



InternConnect Jordan

Intelligent Internship Matching Platform for IT Students

Presented to the Faculty of Computer and Information Technology,
Jordan University of Science and Technology

In Partial Fulfillment of the Requirements for the Degree of Bachelor of **Data Science**

By:

Hamzah Drawsheh

Mahde Hanandeh

Mohammad Othman

Jad Awad-allah

Supervised by:

Dr. Qusai Abu-Ein

Irbid, Jordan

Fall 2026

ABSTRACT:

InternConnect Jordan was developed as a full-stack internship management platform that connects **students**, **companies** and **supervisors** in one secure digital environment. The project aimed to solve common problems in internship coordination, including fragmented application workflows, weak communication between stakeholders, manual supervision, limited visibility into student progress, and the difficulty of matching students with suitable internship opportunities. The platform was implemented as a production-oriented SaaS web application using Next.js, React, TypeScript, Supabase, PostgreSQL Row Level Security, Supabase Storage, and OpenAI-based artificial intelligence services.

The project methodology combined software engineering, database design, role-based access control, and AI-assisted decision support. The system was designed around four main user roles: students, companies, supervisors, and administrators. **Students** can create profiles, upload CVs, browse internships, apply, commit to accepted offers, submit monthly internship reports, and use AI-powered career support. **Companies** can post internships, review applicants, access authorized CV links, communicate with students, and evaluate trainees. **Supervisors** monitor progress, approve workflows, manage users, and review platform analytics. AI services were integrated for internship recommendations, semantic matching, student assistant chat, CV improvement, cover-letter generation, feedback analysis, report-to-skill extraction, company insight summaries, and supervisor analytics.

The final system demonstrates that an internship platform can go beyond basic listing and application management by using AI to improve matching quality, automate repetitive tasks, support student career development, and provide intelligent analytics. The project also showed the importance of production readiness features such as secure APIs, database-level authorization, background jobs, rate limiting, health checks, environment validation, testing, and deployment checklists. Although the system is now suitable for public beta usage, future work should focus on broader automated testing, full observability integration, a separate staging environment, further notification simplification, and scalable vector search infrastructure.

ACKNOWLEDGEMENT:

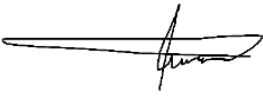
We are very thankful to everyone who supported us complete this paper effectively. We would like to thank our advisor Dr. **Qusai Abu Ein** for his support and instructions that allowed us to organize our project idea and implement it properly, and define the process that we will walk through.

UNDERTAKING:

This is to certify that the graduation project entitled “**InternConnect Jordan: Intelligent Internship Matching Platform for IT Students**” is an original work carried out by the undersigned in partial fulfillment of the requirements for the degree of Bachelor of Data Science at the Department of Computer Information Systems, at the faculty of Computer and Information Technology, Jordan University of Science and Technology.

All phases of the project, including analysis, design, implementation, and system development, were completed by the undersigned. Furthermore, this project has not been submitted, in whole or in part, to any other college or university for the award of any degree or academic qualification.

X



Hamzah Drawsheh

ID:162415

Signed by: trust_cd45affc-7692-4512-bd8d-333b94728407

X



Mahde Hanandeh

ID:163122

Signed by: trust_cd45affc-7692-4512-bd8d-333b94728407

6/14/2026

X



Mohammad Othman

ID:163758

Signed by: trust_cd45affc-7692-4512-bd8d-333b94728407

X



Jad Awad-Allah

ID: 164979

Signed by: trust_cd45affc-7692-4512-bd8d-333b94728407

Table of Contents

<u>PHASE 1</u>	<u>8</u>
1. Introduction	8
1.1 Introduction:.....	8
1.2 Background:.....	9
1.3 Problem statement:	9
1.4 Importance of the project:.....	9
1.5 Required Tools & Technologies:.....	10
2. Literature review.....	11
3. Requirement analysis and specifications	12
3.1 Introduction:.....	12
3.2 Work progress:	12
3.3 Business Requirements identification:.....	13
3.4 User Functional Requirements:	13
3.5 System Functional Requirements:	15
3.6 Non-Functional Requirements and Assumptions	16
3.6.1 Assumptions	16
3.6.2 Non-Functional Requirements	16
4. System Design	17
4.1 Introduction:.....	17
4.2 System Architecture:	17
4.3 Context Diagram:.....	18
4.4 Use case diagram:	19
4.4.1 Sample Use Case Descriptions	19
4.5 Data Flow Diagram:.....	20
4.6 Internship Application Process Flow	21
4.7 Class Diagram.....	22
4.8 Sequence Diagram:.....	23
4.9 Business Process Model:	24
4.9.1 As-Is Process (Current Situation)	24
4.9.2 To-Be Process (Proposed System).....	24
4.10 User interface design:	25

4.10.1 Public Landing Page	25
4.10.2 Authentication and Onboarding Pages.....	26
4.10.3 Student Dashboard.....	26
4.10.4 Internship Discovery and Application Pages	27
4.10.5 Company Dashboard.....	27
4.10.6 Supervisor Dashboard.....	28
4.10.7 Administrator Dashboard.....	28
4.10.8 AI Assistant and Career Tools.....	28
4.11 Site Map:	29
4.12 Data Model:	30
5. Conclusion.....	31
5.1 Conclusion:	31
5.2 Difficulties Faced:	31
5.3 Future Work (Phase 2 Scope):	32
<u>PHASE 2</u>	<u>33</u>
6. System Data Preparation & Database Architecture	33
6.1 Data Preparation and Cleaning.....	33
6.1.1 Schema Normalization and Entity Separation.....	33
6.1.2 Handling Missing and Incomplete Data.....	34
6.1.3 Data Standardization and Encoding.....	34
6.1.4 Outlier and Invalid Record Treatment	34
6.1.5 Migration-Based Data Evolution.....	35
6.2 Platform Data Structure Analysis.....	35
6.2.1 Core Entity Relationships	35
6.2.2 Application Status Distribution (Operational Data)	36
6.2.3 AI-Related Data Structures	36
6.2.4 Security and Access Control over Data	36
6.3 Insights from Data Architecture.....	37
7. Implementation	38
7.1 AI Service Integration	38
7.1.1 Feature Engineering for Recommendations	38
7.1.2 Recommendation Pipeline	39
7.1.3 Large Language Model (LLM) Features	39

7.2 Software Implementation.....	40
7.2.1 Technology Stack	40
7.2.2 Repository Structure	40
7.2.3 GitHub Repository.....	41
7.2.4 README Summary	41
7.2.5 Key Implementation Modules	42
8. Testing & Evaluation	43
8.1 Evaluation Metrics	43
8.1.1 Software Quality Metrics	43
8.1.2 Security Evaluation Metrics	43
8.1.3 Platform Functional Metrics (Post-Launch KPIs)	44
8.1.4 AI Feature Evaluation Criteria	44
8.2 Experimental and Validation Results	44
8.2.1 Production Readiness Scorecard	44
8.2.2 Comparison with Traditional Manual Workflow	45
9. Deployment	45
9.1 Deployment Architecture.....	45
9.1.1 Deployment Components.....	45
9.1.2 Deployment Diagram.....	46
9.2 Maintenance and Scalability	46
9.2.1 Ongoing Maintenance.....	46
9.2.2 Scalability Considerations.....	47
9.2.3 Rollback Strategy	47
10. Results, Discussion & Conclusion	47
10.1 Discussion.....	47
10.1.1 Achievement of Objectives	48
10.1.2 Comparison with Related Work	48
10.1.3 AI Integration Results	48
10.1.4 Limitations	49
10.2 Conclusion and Future Work	49
Future Work	49
References	50

List of Tables

Table 1: Required Tools & Technologies 10

Table 2: Non-Functional Requirements..... 16

Table 3: Schema Normalization and Entity Separation..... 33

Table 4: Application Status Distribution 36

Table 5: AI-Related Data Structure 36

Table 6: LLM Features 39

Table 7: Software Implementation 40

Table 8: GitHub Repo..... 41

Table 9: Key Implementation Models 42

Table 10: Software Quality Metrics 43

Table 11: Security Evaluation Metrics 43

Table 12: Platform Functional Metrics 44

Table 13: AI Feature Evaluation Criteria 44

Table 14: Production Readiness Scorecard..... 44

Table 15: Comparison with Traditional Manual Workflow 45

Table 16: Deployment Components..... 45

Table 17: Ongoing Maintenance 46

Table 18: Scalability Considerations 47

Table 19: Achievement of Objectives 48

List of Figures

Figure 1: System Architecture 17

Figure 2: Context Diagram 18

Figure 3: Use Case Diagram 19

Figure 4: Data Flow 21

Figure 5: Class Diagram 23

Figure 6: Sequence Diagram..... 24

Figure 7: Business Process Model..... 24

Figure 8: Public Landing Page 25

Figure 9: Authentication Page..... 26

Figure 10: Student Dashboard..... 26

Figure 11: Internship Discovery and Application 27

Figure 12: Company Dashboard 27

Figure 13: Supervisor Dashboard..... 28

Figure 14: Site Map 29

Figure 15: Data Model (RLS) 30

Figure 16: Class Diagram 35

Figure 17: Deployment Diagram..... 46

1. Introduction

1.1 Introduction:

Internship training plays a critical role in preparing university students for the labor market, especially in technical and rapidly evolving fields such as Data Science and Artificial Intelligence. Practical training is considered a core graduation requirement, as it provides students with valuable hands-on experience, exposure to real working environments, and an opportunity to apply theoretical knowledge to real-world problems. Through internships, students develop both technical and professional skills that are essential for their future careers.

Despite the importance of internship training, the process of selecting a suitable internship company often lacks proper organization, structure, and transparency. In many cases, students rely on personal communication, informal recommendations, or limited publicly available information when choosing an internship opportunity. This can lead to poor decision-making, mismatches between student specialization and internship tasks, and unequal access to opportunities. Furthermore, the absence of a centralized system makes it difficult for academic supervisors to monitor student progress and evaluate the suitability of internship placements.

To address these challenges, this project proposes a web-based intelligent platform that aims to organize and automate the internship selection and management process. The platform connects students, companies, and academic supervisors within a unified system that ensures transparency and efficiency. In addition, the system integrates an AI-based assistant designed to guide students toward suitable internship opportunities based on their interests, skills, and academic background. By combining structured internship management with intelligent guidance, the proposed system helps students make informed decisions and improves the overall quality of the internship experience.

1.2 Background:

Currently, internship coordination is often handled manually through emails, phone calls, or personal connections. This approach introduces several challenges, including lack of documentation, difficulty in tracking applications, and limited visibility for academic supervisors.

Moreover, students often struggle to identify companies that align with their academic specialization and career goals, which highlights the need for a data-driven system that supports informed decision-making.

1.3 Problem statement:

The absence of an integrated system for managing internship opportunities results in inefficient communication between students and companies. Students lack reliable information about companies, supervisors face challenges in monitoring applications, and companies receive unstructured requests.

Therefore, there is a need for a centralized platform that organizes internship applications and provides intelligent recommendations based on data analysis and AI recommendation systems, this help students and companies to connect each other, and the official university supervisor who will be monitoring the connection.

1.4 Importance of the project:

This project is important as it introduces a structured and data-driven approach to internship placement. By leveraging web technologies and artificial intelligence concepts, the system aims to facilitate the process of finding a training opportunity, save time and money for students, improve decision-making, reduce administrative workload, and enhance the overall internship experience for all stakeholders.

On other side, our service covers the companies and organizations by connecting them with fresh graduate students, our service ensures that company will invest in the right targeted students to make them qualified for the market.

1.5 Required Tools & Technologies:

The platform is designed as a full-stack web application using modern, scalable technologies. The selected technology stack for the project is as follows:

Table 1: Required Tools & Technologies

Layer	Planned Technology	Role
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	User interface, routing, client-side logic
Server logic	Next.js Route Handlers (App Router API routes)	Secure server-side operations, AI integration, email delivery
Database	Supabase (PostgreSQL)	Persistent data storage with Row Level Security (RLS)
Authentication	Supabase Auth	Email/password registration, login, and session management
File storage	Supabase Storage	Student CVs, company logos, and related documents
Artificial Intelligence	OpenAI API	Embeddings, recommendations, chat assistant, CV improvement, cover-letter generation, feedback analysis
Email	SMTP / Resend	Transactional notifications
Development tools	Git, VS Code / Cursor, web browsers, Supabase CLI	Version control, development, and database migration management

Note: The original project proposal referenced a separate Python/FastAPI backend. After architectural analysis during the design phase, the project adopted a **unified Next.js full-stack architecture** backed by Supabase. This approach reduces deployment complexity, improves maintainability, and aligns with contemporary serverless web application design.

2. Literature review

Several studies and existing systems have explored the use of online platforms for internship management and career recommendation services. Traditional internship platforms typically focus on listing available opportunities and providing basic application functionality. While these systems improve accessibility, they often lack intelligent features that support personalized decision-making, academic supervision, or structured evaluation of internship providers.

Platforms such as LinkedIn, Indeed, and university career portals provide large-scale job and internship discovery. However, they generally do not fully support university-specific workflows such as supervisor monitoring, monthly academic reports, employer evaluations, company onboarding approval, and department-scoped supervision. Learning management systems such as Moodle support academic submissions but are not specialized for company-side internship matching, applicant review, or secure CV management.

Recent research emphasizes the effectiveness of data-driven approaches in enhancing user experience and decision support systems. Ranking and matching models based on user profiles, semantic similarity, historical feedback, and relevance to user skills have been shown to improve recommendation quality. Embedding-based similarity search, in particular, enables systems to compare student profiles with internship descriptions beyond simple keyword matching.

Conversational AI and chatbot interfaces have also gained attention as effective tools for assisting users by providing guidance, answering questions, and reducing uncertainty during decision-making processes. Nevertheless, research consistently recommends that AI be used as an **assistive layer** rather than as the sole decision maker, especially in academic and recruitment contexts where fairness, transparency, and human oversight are required.

These findings serve as the foundation for InternConnect Jordan. By integrating structured internship management with intelligent matching mechanisms, an AI-powered student assistant, and supervisor analytics, the platform aims to overcome the limitations of existing solutions. The proposed approach not only facilitates internship applications but also supports students in identifying suitable opportunities aligned with their academic background and career goals, while preserving supervisor and institutional control over the internship process.

3. Requirement analysis and specifications

3.1 Introduction:

This chapter outlines the functional and non-functional requirements of InternConnect Jordan, focusing on user roles, system capabilities, and design assumptions. Requirements were identified through analysis of the current manual internship workflow, review of similar platforms, stakeholder needs, and iterative discussions with the project supervisor.

The platform defines **four primary user roles**: Student, Company, Supervisor, and Admin. Each role has distinct permissions, dashboards, and workflows.

3.2 Work progress:

The project began with understanding the general problem domain and identifying a suitable application area that addresses a real-world challenge faced by students, companies, and academic supervisors during the internship process.

After careful analysis, the team selected the development of a centralized web-based platform connecting **AI and Data Science students** with **technology companies offering internship opportunities in Jordan**. The main objective is to organize the internship application process, reduce manual communication, improve transparency, and support academic supervision throughout the internship lifecycle.

The existing internship process relied heavily on informal communication channels, which made it difficult to track applications, monitor approvals, review monthly reports, and evaluate company performance. These limitations highlighted the need for a unified platform to manage internship applications, onboarding approvals, training progress, evaluations, and reporting in an organized and efficient manner.

Throughout the design phase, the team conducted requirement analysis, system modeling, interface design, architectural planning, and data model specification. The work was carried out iteratively, allowing continuous refinement of requirements and design components based on ongoing feedback.

3.3 Business Requirements identification:

Identifying business requirements is a critical phase in the system development life cycle. An iterative development approach was adopted to allow continuous improvement and refinement of requirements as the project progressed.

Based on stakeholder analysis, the primary business requirements include:

1. Providing a centralized platform for AI and Data Science internship opportunities in Jordan;
2. Enabling students to create profiles, upload CVs, discover internships, apply, and track application status;
3. Enabling companies to register, post internship listings, review applicants, and manage trainee progress;
4. Enabling university supervisors to monitor students within their academic department, review internship reports, and access placement analytics;
5. Enabling administrators to approve company and supervisor onboarding requests, manage users, and moderate platform content;
6. Supporting intelligent internship recommendations and AI-assisted career tools for students;
7. Supporting in-app and email notifications for key workflow events;
8. Supporting direct messaging between authorized users;
9. Collecting structured feedback and evaluations after internship completion to support company reputation indicators;
10. Ensuring data security, role-based access control, and compliance with academic internship requirements.

These business requirements formed the basis for defining detailed functional and non-functional requirements.

3.4 User Functional Requirements:

The following requirements describe the actions each user type may perform in the system.

All Users:

1. All users shall be able to interact with the website using standard web browsers on desktop and mobile devices.
2. All users shall be able to register, log in, and log out securely using email and password.
3. All users shall receive in-app notifications for relevant system events.
4. All users shall be able to manage notification preferences.

Administrator:

5. The administrator shall be able to view and manage all registered users.
6. The administrator shall be able to approve or reject company and supervisor onboarding requests.
7. The administrator shall be able to suspend or restore user accounts when necessary.
8. The administrator shall be able to monitor internship listings, applications, feedback, and platform analytics.
9. The administrator shall be able to access diagnostic and moderation tools for platform maintenance.

Student:

10. The student shall be able to register, verify email, and create a personal profile including university, major, department, skills, and bio.
11. The student shall be able to upload and manage a CV (PDF) and maintain an AI-assisted resume profile.
12. The student shall be able to browse, search, and filter internship listings.
13. The student shall be able to view AI-powered internship recommendations based on profile similarity and skills.
14. The student shall be able to apply to internship opportunities, optionally generating an AI-assisted cover letter.
15. The student shall be able to track application status through the full lifecycle (pending, accepted pending commitment, accepted, rejected, completed, withdrawn).
16. The student shall be able to confirm commitment to an accepted offer within a defined deadline.
17. The student shall be able to submit monthly and weekly internship reports during the training period.
18. The student shall be able to communicate with companies and supervisors through direct messaging.
19. The student shall be able to use an AI chat assistant for platform and career guidance.
20. The student shall be able to use additional AI tools such as CV improvement, interview simulation, and skill-gap insights.
21. The student shall be able to submit post-internship training evaluations.
22. The student shall receive email and in-app notifications when application status changes.

Company Representative

23. The company representative shall be able to register, submit an onboarding request, and create a company profile after admin approval.
24. The company representative shall be able to post, edit, pause, and manage internship listings.
25. The company representative shall be able to view applications submitted to their listings.
26. The company representative shall be able to review student profiles and access authorized CV files through secure links.
27. The company representative shall be able to accept or reject applications with predefined statuses.
28. The company representative shall be able to review trainee monthly reports, attendance, and evaluations.
29. The company representative shall be able to communicate with students through direct messaging.
30. The company representative shall receive notifications when new applications are submitted.

University Supervisor:

31. The university supervisor shall be able to register, submit an onboarding request, and create a supervisor profile after admin approval.
32. The university supervisor shall be able to view students within the same academic department.
33. The university supervisor shall be able to monitor student applications, placements, and internship progress.
34. The university supervisor shall be able to review and approve monthly internship reports.
35. The university supervisor shall be able to access department-level analytics and AI-generated insight summaries.
36. The university supervisor shall be able to export report and placement data for academic use.
37. The university supervisor shall be able to communicate with students through direct messaging.
38. The university supervisor shall receive notifications for key student milestones and report submissions.

3.5 System Functional Requirements:

1. The system shall display a public landing page describing the platform purpose and providing access to login and registration.
2. The system shall support role-based registration for students, with company and supervisor access granted through an admin-approved upgrade workflow.
3. The system shall authenticate users through Supabase Auth and enforce role-based route protection.
4. The system shall provide separate dashboards for students, companies, supervisors, and administrators.
5. The system shall allow authorized users to insert, update, and view data according to their role permissions.
6. The system shall store internship listings with title, description, requirements, location type, duration, deadlines, and status.
7. The system shall record each application individually with timestamps, status, and related internship position.
8. The system shall enforce a commitment workflow whereby an accepted student must confirm placement within a defined period.
9. The system shall ensure that once a student commits to an internship, other active applications are withdrawn according to business rules.
10. The system shall generate in-app notifications and queue transactional emails for key events.
11. The system shall provide an AI recommendation engine using embeddings and deterministic scoring.
12. The system shall provide an AI student assistant accessible from the student dashboard.
13. The system shall support bilingual user interface presentation (English and Arabic).
14. The system shall support dark mode for improved usability.
15. The system shall store student CVs and company logos in secure cloud storage.
16. The system shall maintain an audit-friendly notification and email delivery queue.
17. The system shall compute company reputation indicators based on aggregated training evaluations.
18. The system shall support monthly, weekly, attendance, employer evaluation, and final internship report workflows aligned with university requirements.

3.6 Non-Functional Requirements and Assumptions

3.6.1 Assumptions

- The platform initially targets **AI and Data Science students** and technology-related internship providers in Jordan.
- Each registered company should offer opportunities related to technology, AI, data science, or closely aligned IT fields.
- Each student may apply to multiple companies, but only **one internship placement** may be active at a time after commitment.
- A company may offer multiple internship opportunities simultaneously.
- Application responses use predefined statuses (e.g., pending, accepted, rejected, completed).
- Student feedback and company evaluations are collected according to defined internship lifecycle stages.
- The AI assistant provides advisory support only and does not make final internship decisions.
- The system does not process financial transactions or salary payments.
- Data provided by users is assumed to be accurate; moderation tools are available for administrators.
- Supervisors are matched to students primarily by **academic department** (Computer Science, Computer Information Systems, Software Engineering).
- Company and supervisor accounts require **administrator approval** before full access is granted.

3.6.2 Non-Functional Requirements

Table 2: Non-Functional Requirements

Category	Requirement
Usability	The system shall provide a clear, responsive, and role-appropriate interface for all user types.
Performance	Main pages should load within acceptable time under normal usage; form submissions should be processed without noticeable delay.
Security	User data shall be protected from unauthorized access through authentication, middleware route guards, and Row Level Security.
Availability	The system is intended to be available continuously as a web-based service.
Scalability	The architecture shall support growth in users, listings, applications, and stored documents.
Maintainability	The codebase shall follow modular structure with separated UI components, shared libraries, and SQL migrations.
Portability	The application shall be deployable to standard cloud hosting environments.
Data integrity	Internship, application, and report records shall maintain consistent status transitions.
Internationalization	The interface shall support English and Arabic to serve local users effectively.
Logging and auditability	System activity related to notifications, email delivery, and administrative actions shall be traceable.
Error handling	The system shall handle invalid input, authentication failures, and service interruptions gracefully.
Upgradeability	Components such as AI providers, email services, or storage backends should be replaceable with minimal disruption.

4. System Design

4.1 Introduction:

The main objective of this chapter is to present the overall system design of InternConnect Jordan. This chapter describes the system architecture, interactions between users and system components, and the visual representation of the system through diagrams and interface designs.

Since the graduation project is divided into two phases, **Graduation Project I (Phase 1)** focuses on requirements analysis, architectural design, database modeling, and user interface planning. **Graduation Project II (Phase 2)** will address full implementation, integration testing, deployment, and production hardening. The diagrams and interface designs presented in this chapter represent the approved target design for the platform.

4.2 System Architecture:

The architecture consists of four primary external dependencies accessed through controlled server routes: Supabase Auth, Supabase PostgreSQL, Supabase Storage, and the OpenAI API. Client browsers interact only with the Next.js application, which mediates all sensitive operations and prevents direct exposure of service credentials. Figure1 presents the high-level system architecture. It shows users interacting with the Next.js React frontend, middleware performing role routing, API routes connecting to Supabase and OpenAI services, PostgreSQL enforcing Row Level Security, Storage managing files, and background jobs processing notifications and email queues.

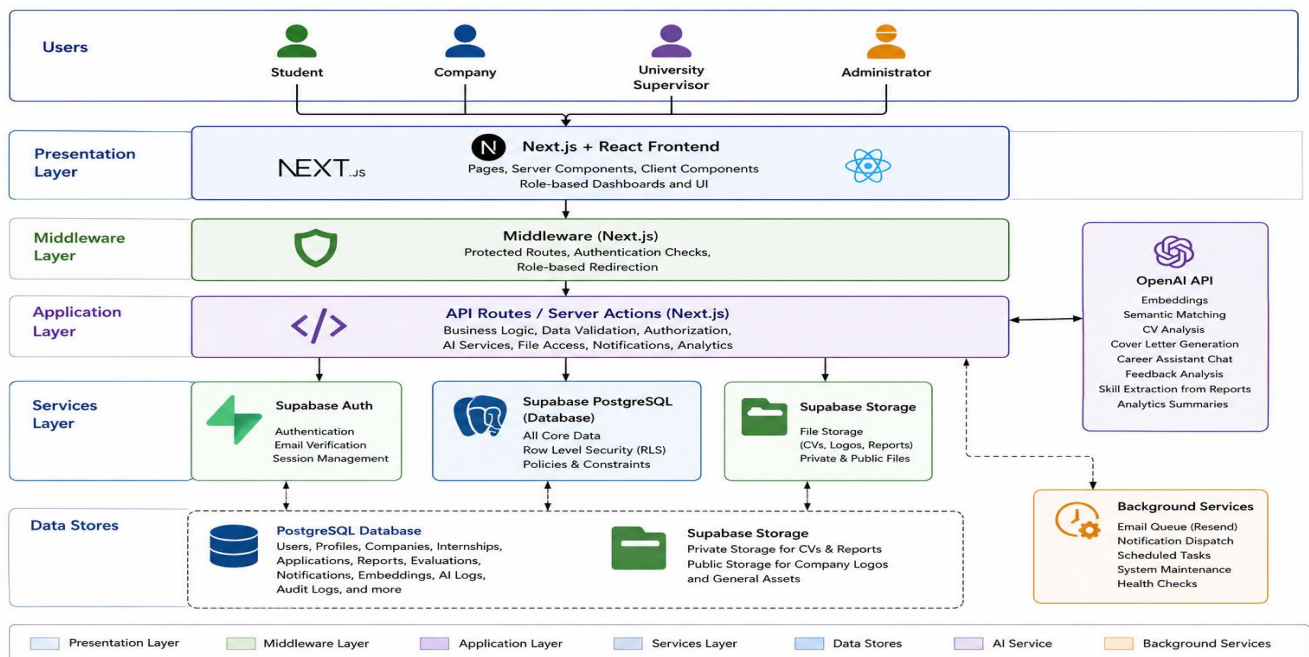


Figure 1: System Architecture

4.3 Context Diagram:

The context diagram represents the system as a single process and illustrates its interaction with external entities. The primary actors are:

- **Student** — discovers internships, applies, submits reports, uses AI tools;
- **Company** — posts listings, reviews applications, evaluates trainees;
- **University Supervisor** — monitors department students and reviews academic reports;
- **Administrator** — approves onboarding, manages users, moderates content;
- **External Services** — OpenAI (AI), email provider (SMTP/Resend), Supabase (auth/data/storage).

Each actor communicates with the system through defined inputs and outputs such as profiles, applications, decisions, reports, notifications, and analytics.

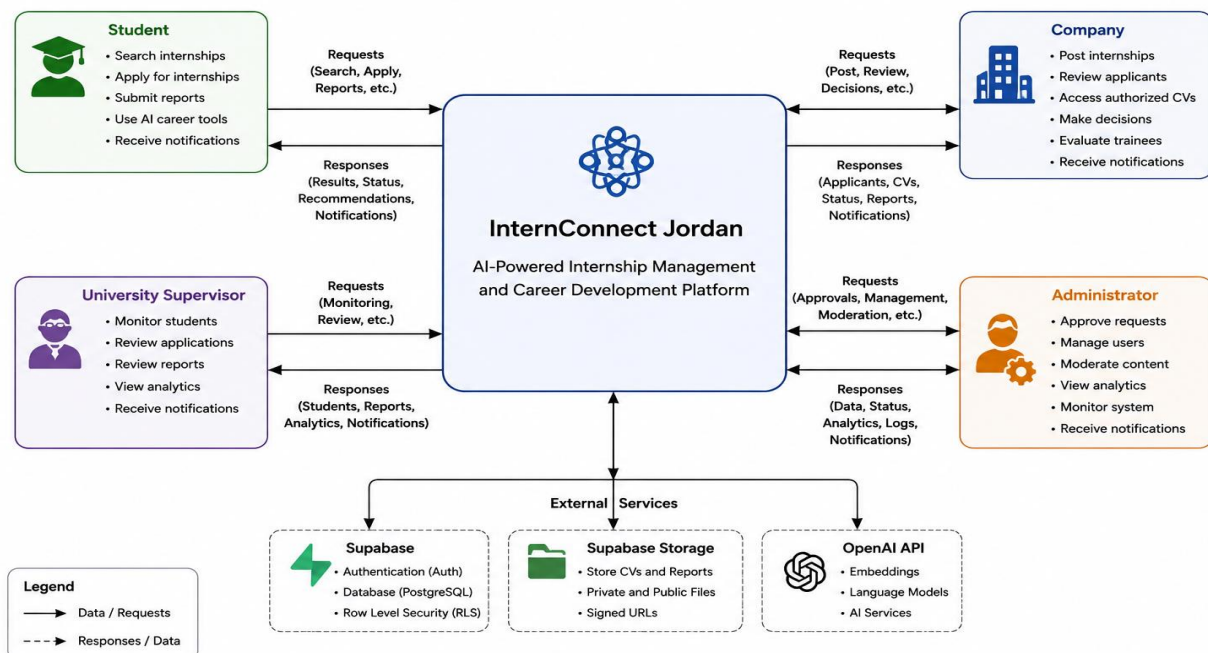


Figure 2: Context Diagram

4.4 Use case diagram:

The use case diagram illustrates the functional behavior of the system from the users' perspectives. It identifies the main actors and the actions they can perform.

Primary Actors: Student, Company, Supervisor, Admin

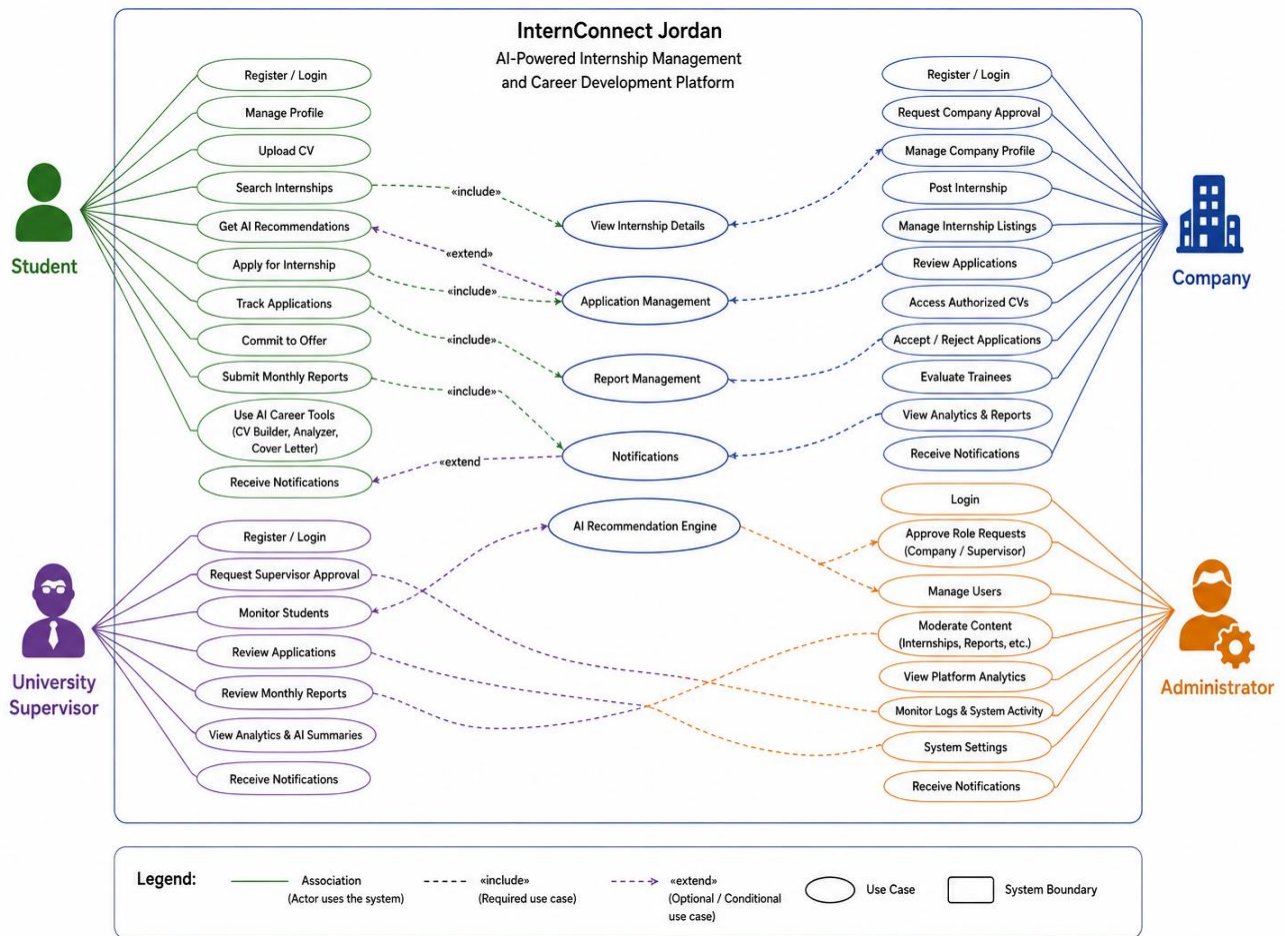


Figure 3: Use Case Diagram

4.4.1 Sample Use Case Descriptions

Use Case 1: Register Account / Log In

Actors: Student, Company, Supervisor

1. User opens the registration or login page.
2. User enters email, password, and required information.
3. System validates input and creates/authenticates the account.
4. User is redirected to the appropriate dashboard or onboarding flow.

Alternative flow: Registration fails due to invalid or duplicate data.

Use Case 2: Apply for Internship

Actor: Student

1. Student logs in and browses available internships.
2. Student selects an internship and reviews details.
3. Student submits an application (optionally with cover letter).
4. System stores the application and notifies the company.

Alternative flow: Application rejected due to deadline expiration or business rule violation.

Use Case 3: Review and Decide on Application

Actor: Company

1. Company logs in and opens the applications panel.
2. Company reviews applicant profile and CV.
3. Company accepts or rejects the application.
4. System updates status and notifies the student and supervisor.

Use Case 4: Confirm Internship Commitment

Actor: Student

1. Student receives an accepted-pending-commitment notification.
2. Student reviews offer details within the allowed timeframe.
3. Student confirms commitment.
4. System finalizes placement and updates related applications.

Use Case 5: Approve Company/Supervisor Onboarding

Actor: Admin

1. Admin reviews pending role upgrade requests.
2. Admin verifies submitted profile information.
3. Admin approves or rejects the request.
4. System updates user role and grants appropriate access.

4.5 Data Flow Diagram:

The Data Flow Diagram (DFD Level 0) provides a high-level overview of how data flows within the Intelligent Internship Matching Platform. It illustrates the interaction between external entities, internal system processes, and data stores. The diagram focuses on core processes such as authentication, internship application handling, company responses, and supervisor monitoring.

The Data Flow Diagram (Level 0) provides a high-level overview of how data flows within InternConnect Jordan. It illustrates the interaction between external entities, internal processes, and data stores.

Core Processes:

1. User authentication and session management;
2. Profile and document management;
3. Internship listing management;
4. Application submission and decision handling;
5. Commitment and training lifecycle management;
6. Internship report submission and review;
7. Notification and email dispatch;
8. AI recommendation and assistant processing;
9. Administrative moderation and analytics.

Primary Data Stores:

- User profiles and role data;
- Student, company, and supervisor profiles;
- Internship positions and applications;
- Training records and reports;
- Notifications and email queue;
- Messages and evaluations.

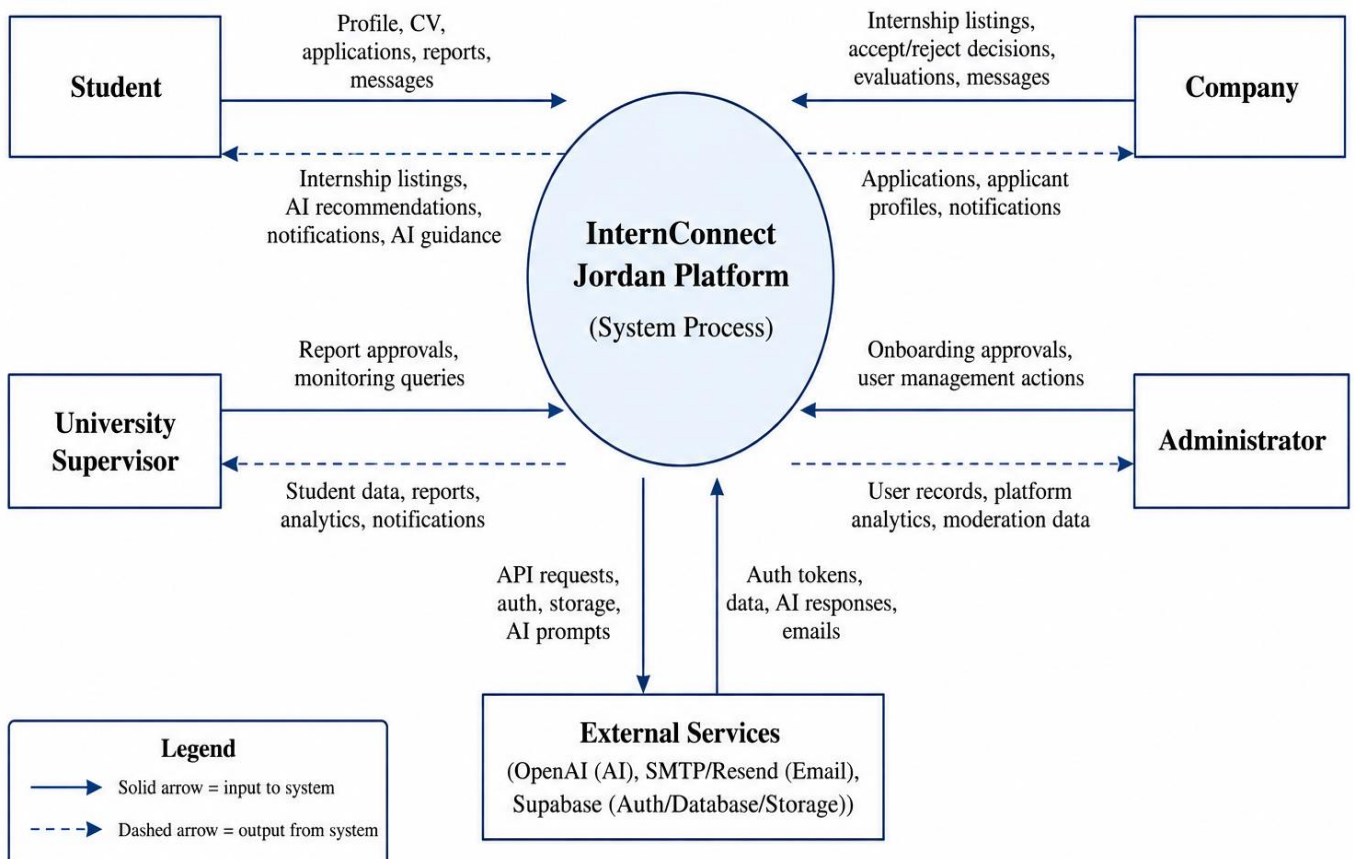


Figure 4: Data Flow

4.6 Internship Application Process Flow

1. The student accesses the platform and authenticates successfully.
2. The system retrieves the student profile and displays available internships, including AI-ranked recommendations.
3. The student selects an internship and submits an application.
4. The system validates the application and stores it in the applications data store.
5. The company receives a notification and reviews the application.
6. The company accepts or rejects the application; the system updates the status.
7. If accepted, the student enters a commitment window and confirms the placement.
8. The system creates an active training record and notifies the student, company, and supervisor.
9. During training, the student submits monthly reports; the company and supervisor review them.
10. Upon completion, evaluations are collected and company reputation indicators are updated.

4.7 Class Diagram

The object-oriented class diagram defines the logical structure of the system and supports future database implementation.

Core Classes:

- **User / Profile** — common attributes: user ID, name, email, role, account status;
- **Student** — extends user with university, major, department, skills, CV path, preferences, embedding;
- **Company** — extends user with company name, description, location, logo, evaluation metrics;
- **Supervisor** — extends user with department, title, university;
- **InternshipPosition** — title, description, requirements, dates, deadline, status, embedding;
- **Application** — application date, status, commitment fields, related student and position;
- **Internship (Training Record)** — active placement linked to an accepted application;
- **InternshipReport** — monthly/weekly report content, status, signatures;
- **Notification** — type, message, read status, recipient;
- **Message** — direct communication between authorized users;
- **Evaluation** — post-training feedback and rating dimensions.

Key Relationships:

- A student may submit multiple applications; each application relates to one internship position;
- A company may publish multiple internship positions;
- An accepted and committed application creates one active training record;
- A training record may have multiple reports and evaluations;
- A supervisor is associated with students through department alignment.

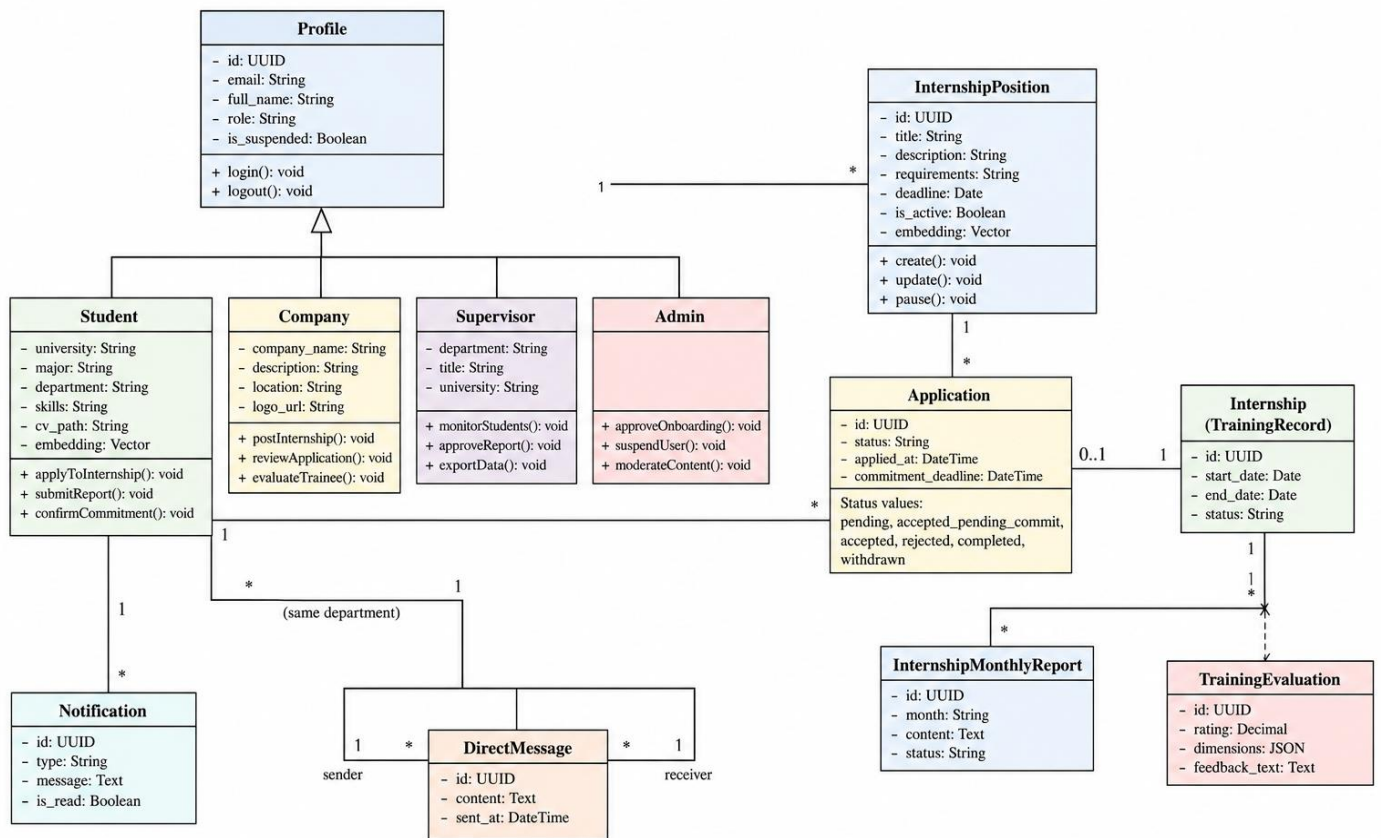


Figure 5: Class Diagram

4.8 Sequence Diagram:

The sequence diagram illustrates the dynamic interaction between actors and system components during a typical application and decision workflow.

1. Student logs in; system authenticates via Supabase Auth.
2. Student requests internship listings; system returns filtered and recommended results.
3. Student submits an application; system stores data and creates a notification.
4. Company views the application and submits a decision.
5. System updates application status and notifies student and supervisor.
6. Student confirms commitment; system finalizes placement.
7. Supervisor views updated placement status on the monitoring dashboard.

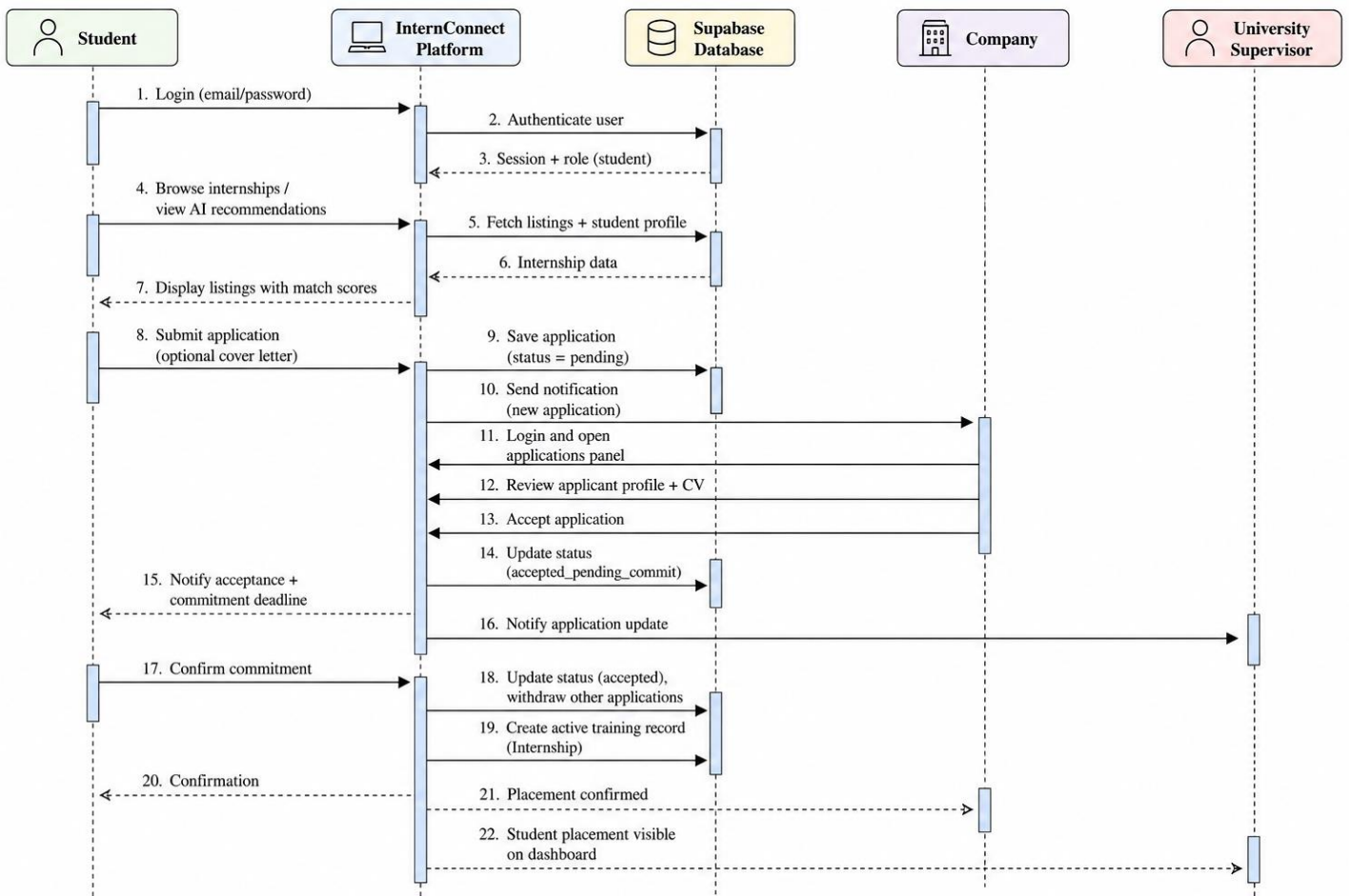


Figure 6: Sequence Diagram

4.9 Business Process Model:

4.9.1 As-Is Process (Current Situation)

- The student searches for internship opportunities through personal communication or informal channels.
- The student contacts companies manually via email or phone.
- Internship requests are handled without a unified system or documentation.
- The academic supervisor has limited visibility over student applications.
- Tracking application status is difficult and time-consuming.

4.9.2 To-Be Process (Proposed System)

- The student logs into the internship platform and views available companies.
- Internship applications are submitted electronically through the system.
- Companies receive structured applications and respond directly through the platform.
- The system automatically updates application status and notifies all parties.
- The academic supervisor monitors applications in real time.
- The process becomes centralized, transparent, and efficient.

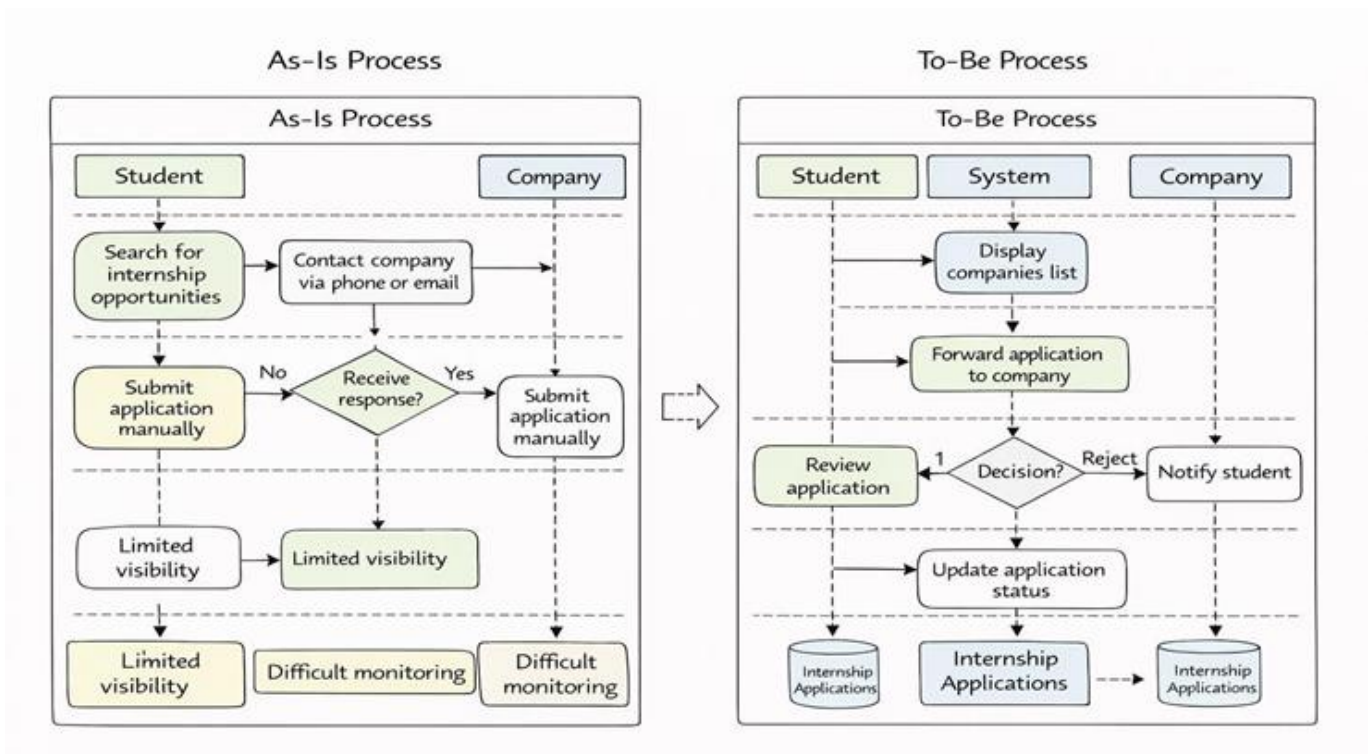


Figure 7: Business Process Model

4.10 User interface design:

This section presents the user interface design in the form of a storyboard. The interface is designed to be simple, intuitive, responsive, and role-aware. The platform supports **English and Arabic** and includes **dark mode** for improved accessibility.

4.10.1 Public Landing Page

- Navigation to Login, Register, and About sections;
- Overview of platform value for students, companies, and supervisors;
- Call-to-action buttons for registration;
- Platform statistics and feature highlights.

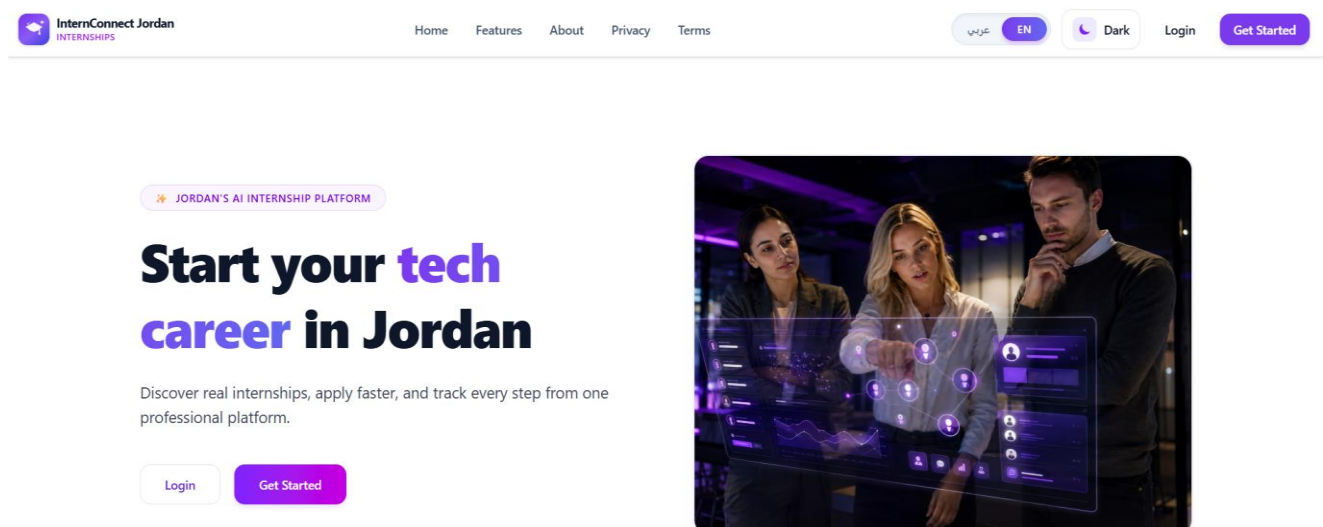


Figure 8: Public Landing Page

4.10.2 Authentication and Onboarding Pages

- Login form (email and password);
- Registration with role selection;
- Email verification flow;
- Company and supervisor onboarding forms;
- Pending approval screen for accounts awaiting admin review.

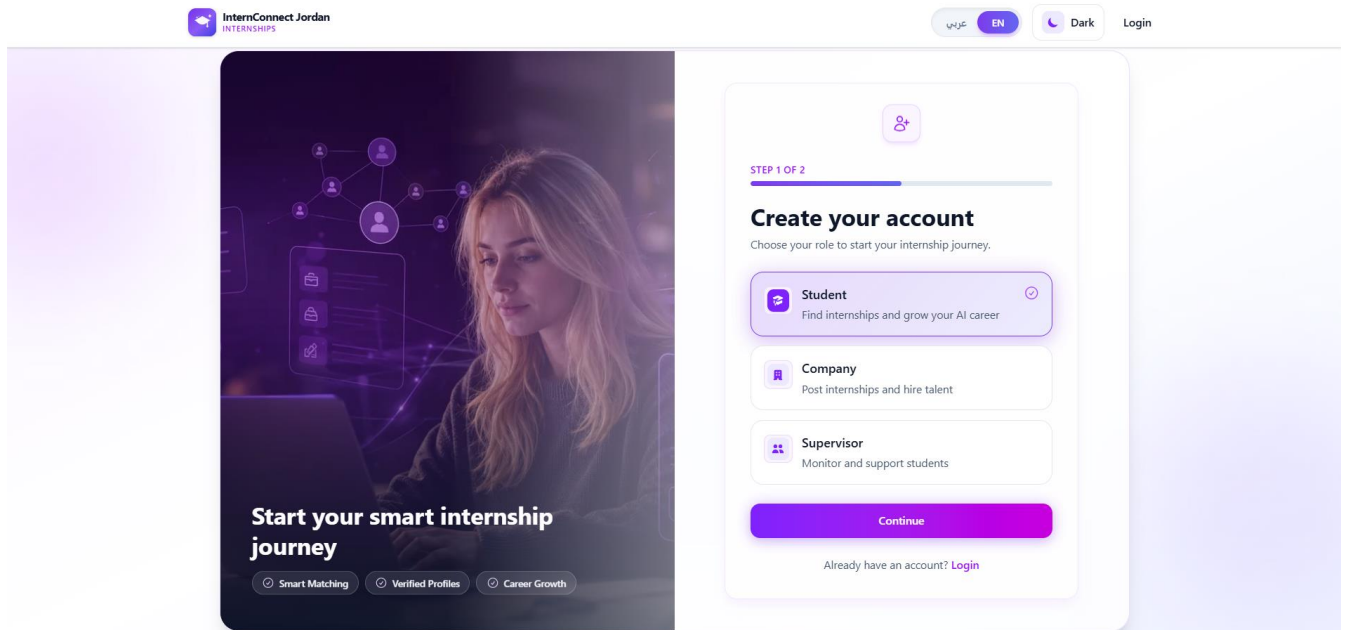


Figure 9: Authentication Page

4.10.3 Student Dashboard

- Summary widgets: active applications, recommendations, training progress;
- AI assistant entry point;
- Quick links to internships, applications, profile, resume builder, messages, and reports;
- Notification center.

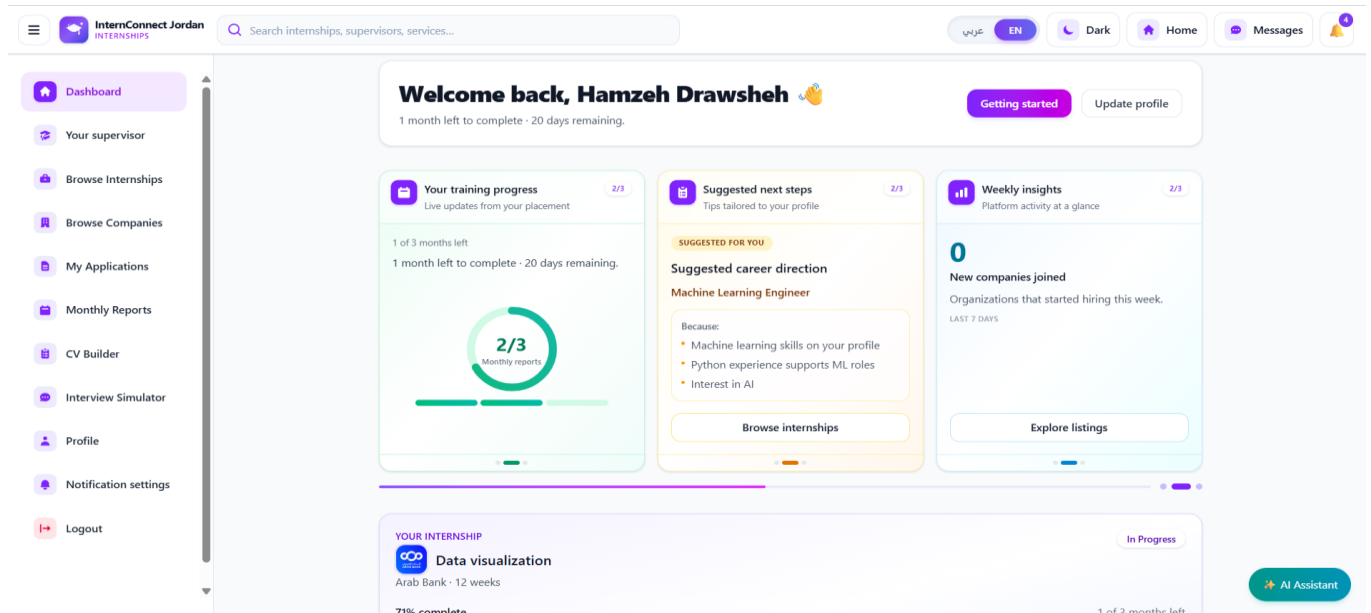


Figure 10: Student Dashboard

4.10.4 Internship Discovery and Application Pages

- Searchable and filterable internship list;
- AI recommendation panel with match indicators;
- Internship detail page with requirements and company information;
- Apply button with optional AI-generated cover letter;
- Application confirmation and status tracking.

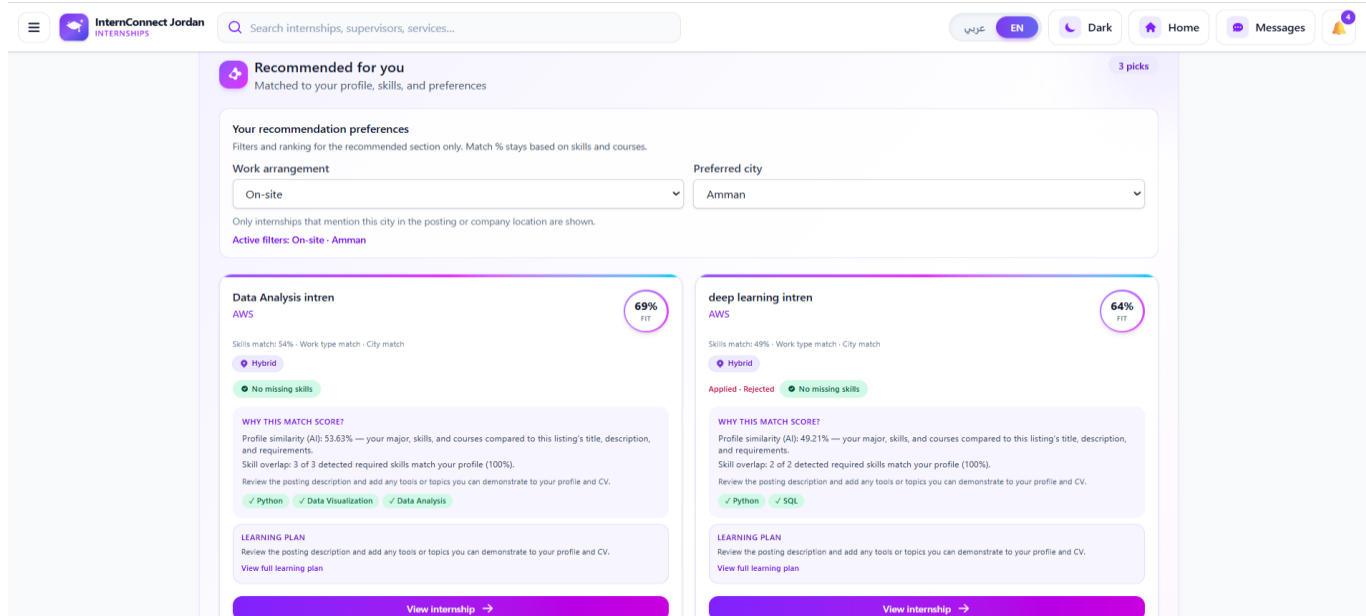


Figure 11: Internship Discovery and Application

4.10.5 Company Dashboard

- Widgets for listings, applications, trainee progress, and reputation indicators;
- Internship management (create, edit, pause, close);
- Applicant review panel with secure CV access;
- Trainee report and evaluation management;
- Messaging interface.

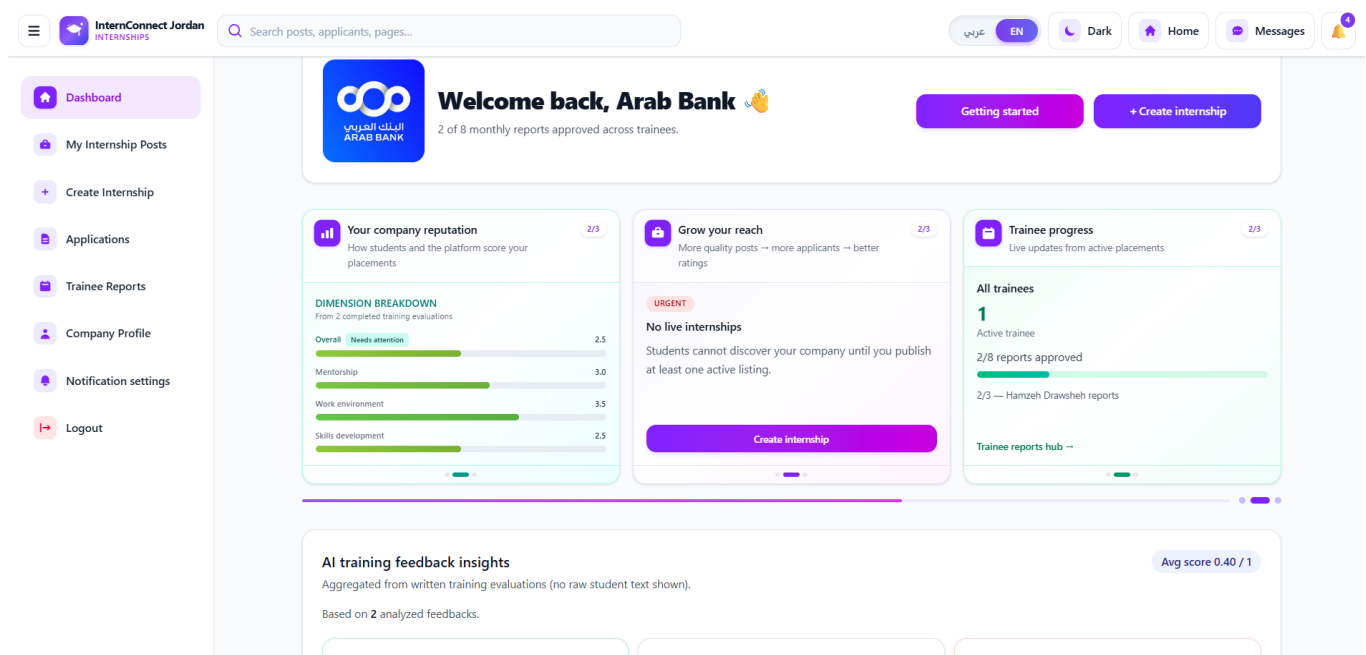


Figure 12: Company Dashboard

4.10.6 Supervisor Dashboard

- Department overview and student list;
- Application and placement monitoring;
- Internship report review and approval;
- AI-generated department insights;
- Export and reporting tools.

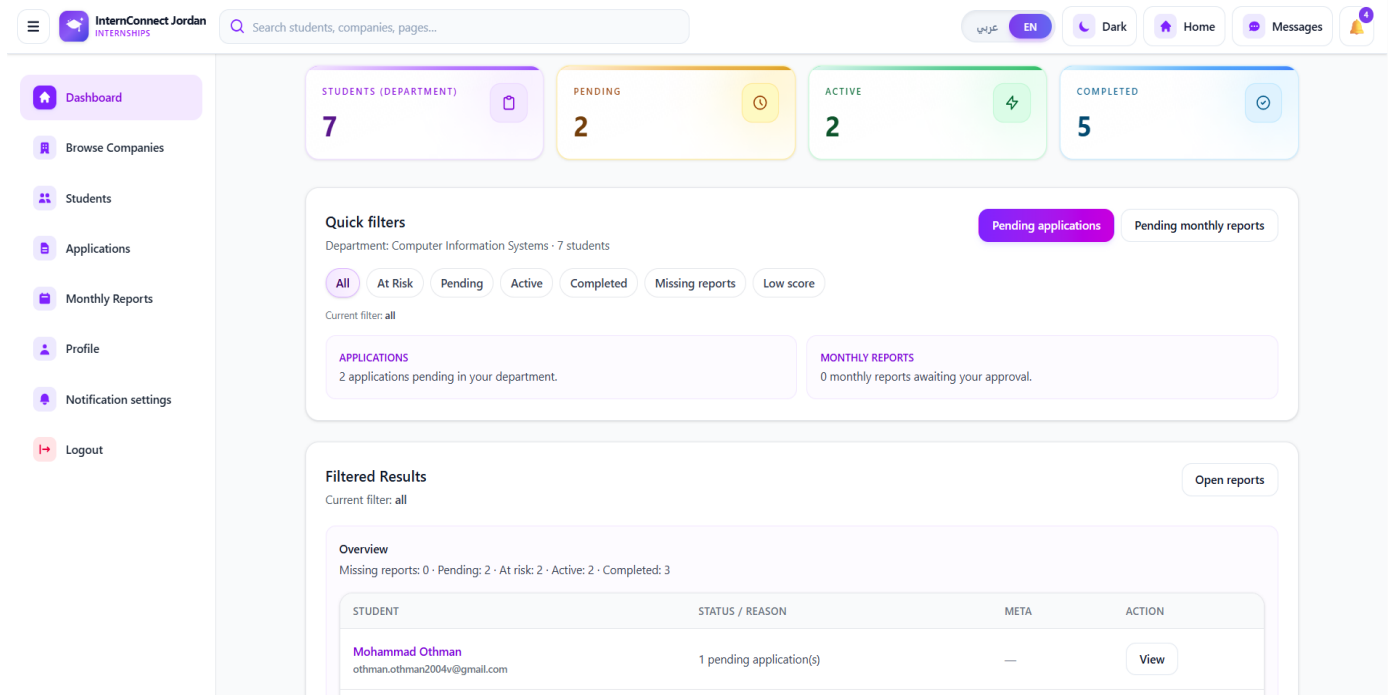


Figure 13: Supervisor Dashboard

4.10.7 Administrator Dashboard

- User management and account suspension tools;
- Onboarding approval queue;
- Internship and application oversight;
- Feedback, report, and analytics views;
- Platform moderation utilities.
-

4.10.8 AI Assistant and Career Tools

- Chat window for student guidance;
- Resume builder with AI improvement suggestions;
- Cover letter generator;
- Interview simulator (planned interface integration);
- Skill-gap and learning plan displays on internship detail pages.

4.11 Site Map:

The site map organizes the main pages according to user role and access level.

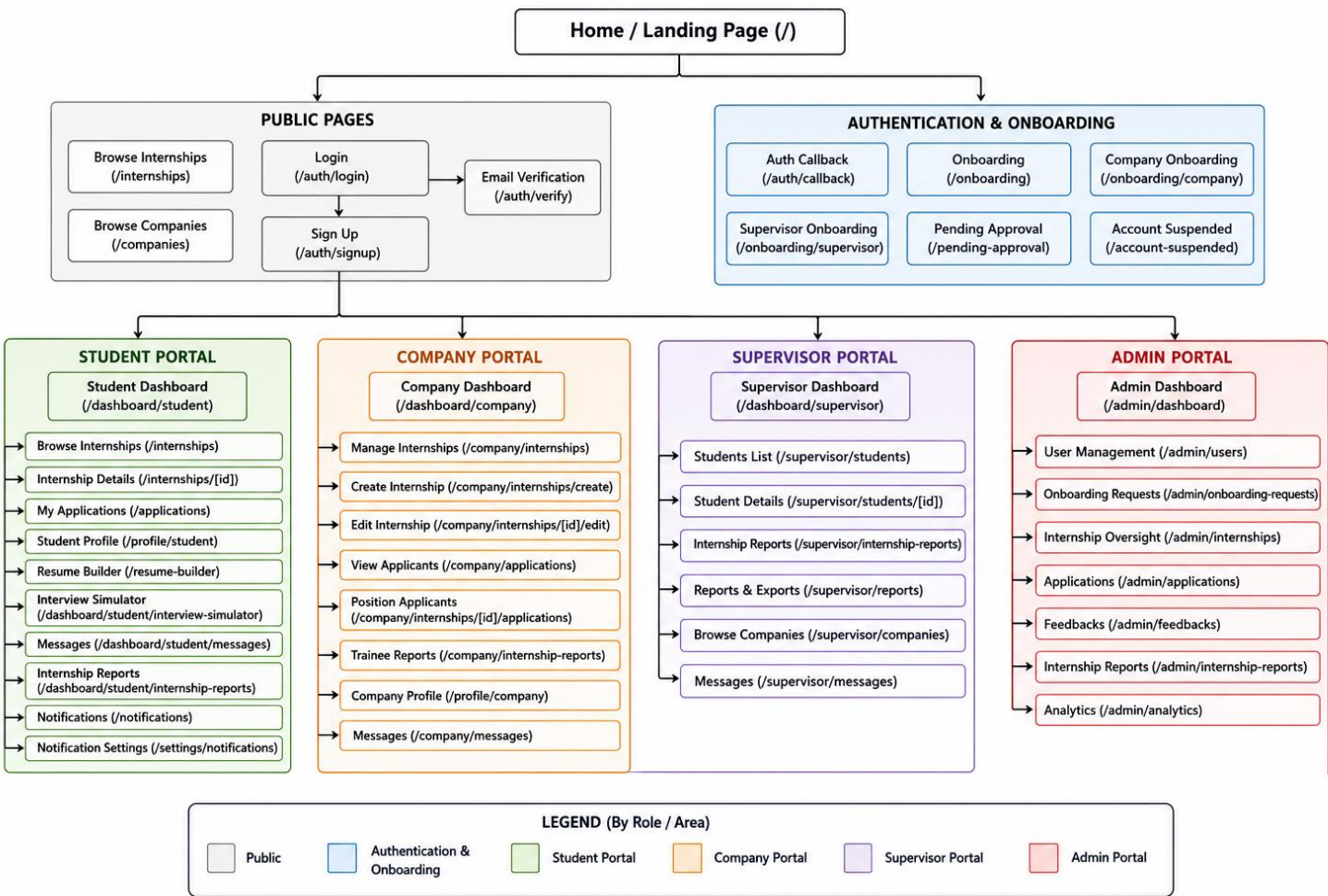


Figure 14: Site Map

4.12 Data Model:

The data model defines the database structure for the platform. The canonical schema is expressed through Supabase SQL migrations and enforces Row Level Security (RLS) on sensitive tables.

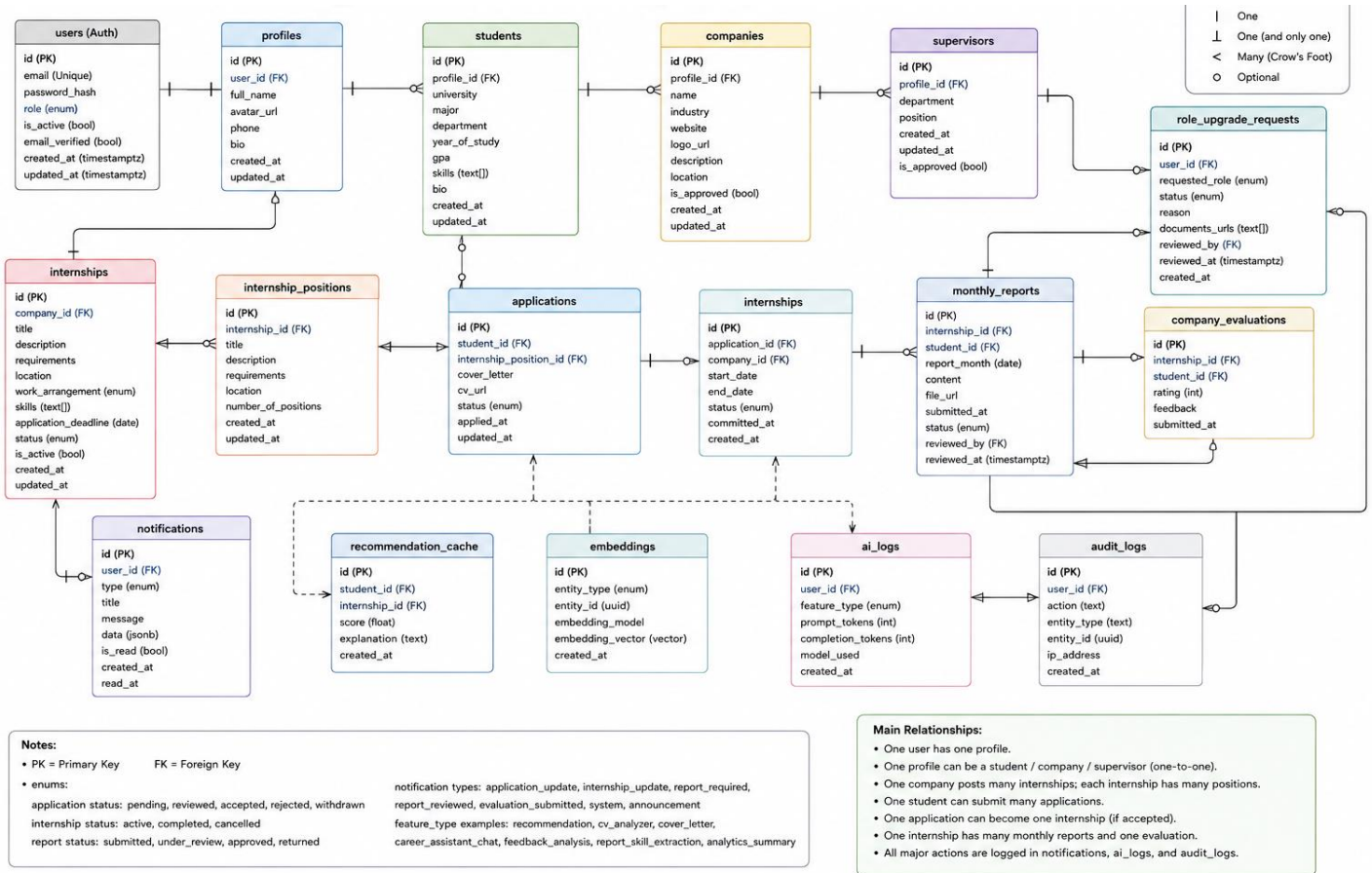


Figure 15: Data Model (RLS)

Application Status Lifecycle: pending → accepted_pending_commit → accepted → completed

Additional states: rejected, withdrawn, commitment_expired

Security Design:

- Supabase Auth for identity management;
- Middleware for route-level role enforcement;
- PostgreSQL Row Level Security for table-level access control;
- Server-side API routes for operations requiring secrets or service-role access.

5. Conclusion

5.1 Conclusion:

This report presented the design and planning of **InternConnect Jordan**, a web-based intelligent internship management platform aimed at supporting AI and Data Science students in Jordan. The system was conceived to address the limitations of the traditional internship process by providing a centralized, structured, and transparent platform for managing the full internship lifecycle—from discovery and application through supervision, reporting, and evaluation.

Through requirement analysis and system design, the proposed solution enables effective interaction among students, companies, university supervisors, and administrators. The platform improves accessibility to relevant opportunities, simplifies application procedures, increases visibility for supervisors, and lays the foundation for AI-assisted matching and career guidance. The adopted architecture—combining Next.js, Supabase, and OpenAI services—provides a scalable and maintainable foundation for the implementation phase.

Overall, this phase demonstrates how structured analysis and technology-driven design can improve academic and professional coordination processes while supporting future enhancements in automation, analytics, and intelligent decision support.

5.2 Difficulties Faced:

During the design and planning phase, the team encountered several challenges:

- Understanding and analyzing the existing manual internship workflow and identifying its operational gaps;
- Reconciling initial architectural assumptions (separate Python backend) with a more maintainable full-stack Next.js and Supabase design;
- Gathering accurate information about comparable platforms and selecting appropriate database structures for multi-role workflows;
- Designing clear and consistent relationships among users, applications, training records, and reports;
- Specifying AI-assisted features in a way that supports students without compromising fairness or supervisor authority;
- Coordinating tasks among team members while maintaining consistent documentation and design artifacts;
- Designing role-based interfaces that serve different stakeholders while preserving a unified user experience;
- Defining department-scoped supervisor access and academic report workflows aligned with university expectations.

5.3 Future Work (Phase 2 Scope):

The following activities are planned for **Graduation Project II (Phase 2)** and are intentionally excluded from this Phase 1 report:

Implementation

- Full implementation of the Next.js application and Supabase database schema;
- Integration of Supabase Auth, Storage, and Row Level Security policies;
- Development of all API routes for AI services, email delivery, PDF generation, and administrative operations;
- Implementation of direct messaging, notification systems, and report workflows;
- Integration of OpenAI services for recommendations, chat assistant, CV improvement, cover letters, and analytics.

Testing

- Unit and integration testing of critical workflows;
- Authorization and security testing for role-based access;
- User acceptance testing with representative student, company, and supervisor scenarios;
- Performance and load testing for core pages and API endpoints.

Deployment

- Production deployment to Vercel;
- Supabase project configuration and migration deployment;
- Environment validation, health checks, and monitoring setup;
- Public launch checklist execution and post-deployment smoke testing.

Additional Enhancements (Post-Launch)

- Expanded automated testing and observability;
- Separate staging environment;
- Mobile application support;
- Video interview scheduling integration;
- Multi-university customization;
- Advanced vector search infrastructure for large-scale recommendation workloads.

End of Graduation Project I (Phase 1) Report

Implementation, Testing & Deployment — to be documented in Graduation Project II (Phase 2)

6. System Data Preparation & Database Architecture

This chapter documents how platform data was structured, validated, and prepared for implementation. Unlike traditional data-science projects that rely on external datasets and exploratory data analysis (EDA), InternConnect Jordan generates and manages its own operational data through user registration, profile creation, internship postings, applications, reports, and evaluations. Therefore, this chapter focuses on **database schema design, data normalization, migration strategy, and data-quality controls** rather than statistical EDA on a static dataset.

6.1 Data Preparation and Cleaning

Data preparation in this project refers to ensuring that all stored records are **consistent, validated, and secure** before and during system operation. The following steps were applied during implementation.

6.1.1 Schema Normalization and Entity Separation

The database was normalized into role-specific and workflow-specific tables rather than storing all information in a single user record. Core entities include:

Table 3: Schema Normalization and Entity Separation

Entity	Table	Purpose
User identity	profiles	Links Supabase Auth users to platform roles
Student profile	students	University, department, skills, CV path, embedding
Company profile	companies	Company name, description, logo, evaluation fields
Supervisor profile	supervisors	Department, title, university
Internship listing	internship_positions	Posted opportunities with requirements and embeddings
Application	applications	Student-to-position applications and status lifecycle
Active training	internships	Placement record after student commitment
Monthly report	internship_monthly_reports	Academic internship reporting workflow

Justification: Separating profiles by role prevents redundant fields, supports Row Level Security (RLS) policies per table, and aligns with the four-role architecture of the platform.

6.1.2 Handling Missing and Incomplete Data

Missing or incomplete data is handled at three levels:

1. **Frontend validation** — Required fields (email, password, university, department, internship title, application deadline) are validated before submission.
2. **Database constraints** — `NOT NULL`, foreign keys, check constraints, and enum-like status fields enforce valid records.
3. **Business rules in application logic** — For example, a student cannot apply without a completed profile; a company cannot review applicants until onboarding is approved by an administrator.

Example:

If a student profile lacks a CV path, the application flow still permits browsing internships but prompts the student to upload a CV before submitting critical applications. Optional AI features (cover-letter generation, recommendations) degrade gracefully when profile fields are incomplete rather than storing invalid records.

6.1.3 Data Standardization and Encoding

Several standardization rules were implemented:

- **Academic departments** are normalized to three canonical values: Computer Science, Computer Information Systems, and Software Engineering (`lib/departments.ts` and SQL migrations).
- **Application statuses** use predefined values: `pending`, `accepted_pending_commit`, `accepted`, `rejected`, `completed`, `withdrawn`, `commitment_expired`.
- **Role values** are restricted to: `student`, `company`, `supervisor`, `admin`.
- **Embeddings** are stored as PostgreSQL `vector(1536)` values generated from standardized text representations of student and internship profiles.

Justification: Standardized categorical values prevent inconsistent filtering, enable reliable supervisor–student department matching, and support predictable workflow automation.

6.1.4 Outlier and Invalid Record Treatment

Rather than statistical outlier removal, the system rejects or flags **invalid operational records**:

- Duplicate applications to the same internship are blocked at the database/application layer.
- Expired internship listings are deactivated through scheduled maintenance (`auto_complete_expired_trainings`, listing expiry logic).
- Suspended users are blocked by middleware and profile flags (`is_suspended`).
- Unauthorized role changes are prevented by a database trigger on protected `profiles` columns.

Example:

When a student confirms commitment to an accepted internship, other active applications are automatically withdrawn according to business rules, ensuring only one active placement exists at a time.

6.1.5 Migration-Based Data Evolution

All schema changes were managed through **83 versioned SQL migrations** in `frontend/supabase/migrations/`. This ensures reproducible database setup across development and production environments.

Deployment command:

```
cd frontend
npm run supabase:push
```

6.2 Platform Data Structure Analysis

Because the project does not use an external training dataset, this section presents the **logical data structure and key relationships** that guided implementation, replacing traditional EDA charts and heatmaps.

6.2.1 Core Entity Relationships

The platform data model centers on the following relationships:

- One profile maps to at most one role-specific record (students, companies, or supervisors).
- One company publishes many internship_positions.
- One student submits many applications, each linked to one internship_position.
- One accepted and committed application creates one active internship (training record).
- One internship contains many internship monthly reports, weekly reports, and evaluations.

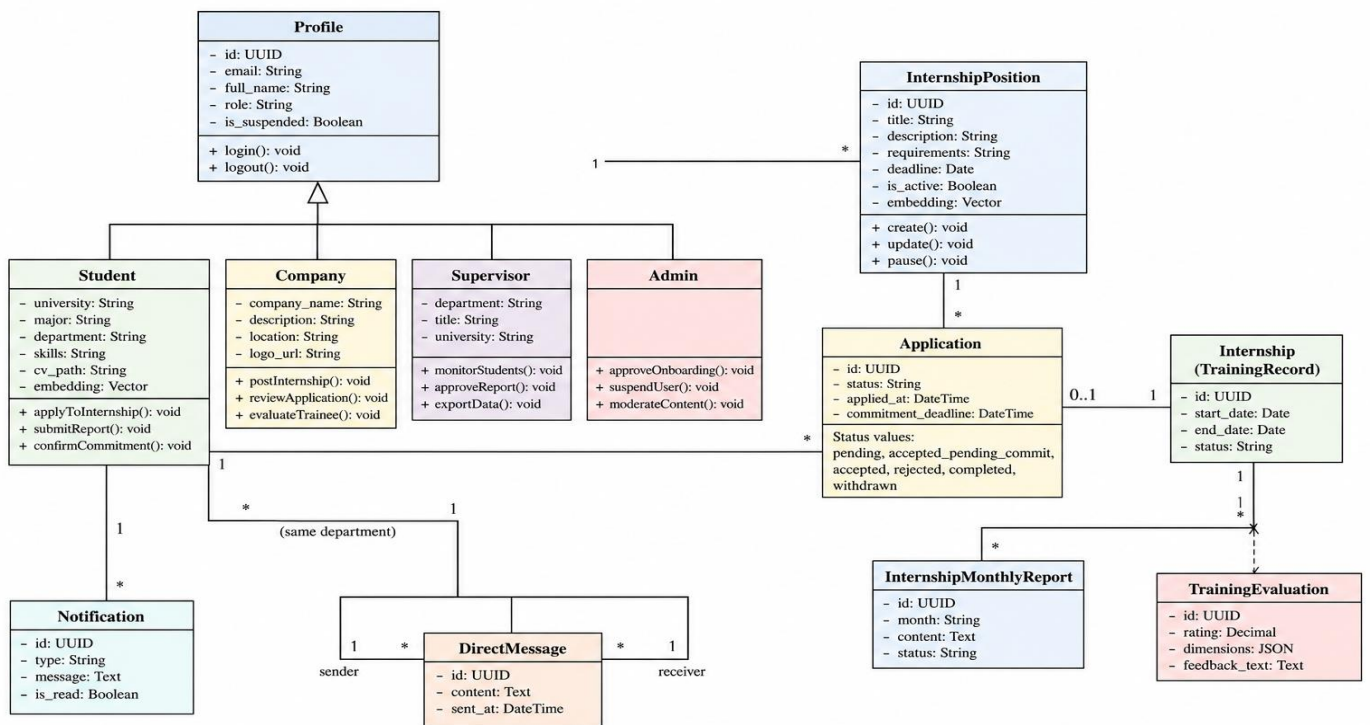


Figure 16: Class Diagram

6.2.2 Application Status Distribution (Operational Data)

The system tracks application flow through predefined statuses rather than free-text states. This design supports dashboards, notifications, and supervisor monitoring.

Table 4: Application Status Distribution

Status	Meaning
<code>pending</code>	Submitted; awaiting company review
<code>accepted_pending_commit</code>	Company accepted; student must confirm within deadline
<code>accepted</code>	Student committed; training active
<code>rejected</code>	Company rejected the application
<code>completed</code>	Internship/training finished
<code>withdrawn</code>	Student or system withdrew the application
<code>commitment_expired</code>	Student did not confirm within allowed window

6.2.3 AI-Related Data Structures

AI features rely on structured platform data rather than external labeled datasets:

Table 5: AI-Related Data Structure

Data Element	Storage	Used By
Student embedding	<code>students.embedding</code>	Internship recommendations
Internship embedding	<code>internship_positions.embedding</code>	Semantic matching
Recommendation cache	<code>student_recommendation_cache</code>	Performance optimization
Feedback analysis	<code>feedback_analysis</code>	Sentiment and summary extraction
Report-derived skills	<code>student_report_skills</code>	Skill extraction from monthly reports

6.2.4 Security and Access Control over Data

Data visibility is enforced through **Row Level Security (RLS)** policies:

- Students can read/write their own profiles, applications, and reports.
- Companies can access applications and trainee data only for their own listings.
- Supervisors can access students within the same academic department.
- Administrators use elevated service-role APIs for moderation and diagnostics.

This access model was as important as schema design because internship data includes CVs, evaluations, and personal academic records.

6.3 Insights from Data Architecture

The following insights from data preparation and schema analysis directly influenced engineering decisions:

- 1. Department-based supervisor matching** — Analysis of university workflow requirements showed that supervisors monitor students by academic department, not only by individually assigned IDs. This led to department normalization and department-scoped RLS policies.
- 2. Commitment workflow requirement** — Operational rules required that a student may hold only one active placement at a time. This insight led to the `accepted_pending_commit` status and automatic withdrawal of competing applications upon commitment.
- 3. Embedding-based matching without ANN indexing** — Review of Supabase `pgvector` capabilities showed that approximate nearest-neighbor indexing was not available in the current setup. This led to application-level cosine similarity scoring plus a recommendation cache table to reduce repeated computation.
- 4. Separation of notifications and email queue** — Data flow analysis revealed that in-app notifications and outbound emails serve different purposes. The design uses `notifications` for user-visible events and `transactional_email_queue` for delivery tracking and retries.
- 5. Company reputation as derived data** — Aggregated training evaluations are stored and computed rather than entered manually, enabling reputation tiers (white / gray / black) used in recommendation ranking.

Example insight statement:

Analysis of the internship lifecycle revealed that application status transitions are the most critical data structure in the system. This led to strict status enums, timestamp fields, and notification triggers at each transition rather than unstructured status text fields.

7. Implementation

This chapter demonstrates technical competence in full-stack software development, AI service integration, database engineering, and reproducible development using Git version control.

7.1 AI Service Integration

InternConnect Jordan does not train custom machine-learning models on a local dataset. Instead, it integrates **OpenAI embedding and language models** as cloud services, combined with deterministic application logic for ranking and workflow control. This section replaces traditional "model training" with **AI service configuration and integration**.

7.1.1 Feature Engineering for Recommendations

Feature engineering refers to transforming raw profile and listing text into structured inputs for similarity matching and scoring.

Student embedding inputs (from `lib/ai/embeddings.ts`):

- University, major, department
- Technical and soft skills
- CV summary, experience, projects
- GPA, courses, custom courses
- Preferences (location type, work arrangement)

Internship embedding inputs:

- Title and work arrangement
- Description and requirements

Additional deterministic features:

- Skill-gap analysis via token/synonym matching (`lib/skill-match.ts`)
- Location and work-type preference alignment (`lib/recommendations/location-prefs.ts`)
- Company reputation score from aggregated evaluations (`lib/companies/evaluation.ts`)

Example:

A student's department ("Computer Information Systems"), listed skills ("Python, SQL, Machine Learning"), and CV project descriptions are concatenated into a normalized text profile before generating a 1536-dimensional embedding using OpenAI `text-embedding-3-small`.

7.1.2 Recommendation Pipeline

The recommendation engine combines semantic and rule-based scoring:

1. Generate or retrieve student and internship embeddings.
2. Compute cosine similarity in Node.js (`lib/ai/vector-utils.ts`).
3. Apply location/work-type filters.
4. Compute skill-gap scores.
5. Blend company reputation indicators.
6. Rank and return top matches.
7. Cache results in `student_recommendation_cache` (1-hour TTL).

API endpoint: `GET /api/recommendations/internships`

This hybrid design was selected because AI similarity alone cannot enforce academic rules, commitment constraints, or supervisor visibility requirements.

7.1.3 Large Language Model (LLM) Features

Table 6: LLM Features

Feature	API Route	Model (configurable)	Purpose
Student assistant chat	<code>/api/chat/student-assistant</code>	gpt-4o-mini (default)	Career and platform guidance
CV improvement	<code>/api/resume/improve</code>	OPENAI_RESUME_MODEL	Suggest profile/CV improvements
Cover letter generator	<code>/api/ai/cover-letter</code>	OPENAI_COVER_LETTER_MODEL	Personalized application letters
Task-to-skill extraction	<code>/api/ai/task-to-skill</code>	OPENAI_TASK_TO_SKILL_MODEL	Extract skills from report tasks
Feedback analysis	<code>/api/feedback/analyze</code>	OPENAI_FEEDBACK_MODEL	Sentiment and keyword analysis
Interview simulator	<code>/api/ai/interview-simulator</code>	OPENAI_INTERVIEW_MODEL	Mock interview Q&A

All AI routes include **rate limiting** (`lib/server/in-memory-user-rate-limit.ts` and Supabase-backed limits in production) to control cost and prevent abuse.

7.2 Software Implementation

7.2.1 Technology Stack

Table 7: Software Implementation

Layer	Technology
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4
Backend	Next.js Route Handlers (frontend/app/api/)
Database	Supabase PostgreSQL + RLS
Authentication	Supabase Auth
Storage	Supabase Storage (CVs, logos, report PDFs)
AI	OpenAI API (embeddings + chat/completions)
Email	Nodemailer (SMTP) + Resend fallback
Testing	Vitest, ESLint, Next.js build
Deployment	Vercel + Supabase

7.2.2 Repository Structure

```
Intrenships_Platform-1/
├── frontend/                                # Full-stack application (implementation)
│   ├── app/                                # Pages + 27 API routes
│   ├── components/                         # UI components by domain
│   ├── lib/                               # Supabase, AI, auth, email, i18n
│   ├── supabase/migrations/               # 83 SQL migrations (schema source of truth)
│   ├── middleware.ts                      # Auth + role-based route protection
│   ├── vitest.config.ts                   # Test configuration
│   └── package.json
├── CONTEXT_ENG/                           # Project documentation
├── Project Diagrams/                       # Report diagrams
└── README.md
```

Note on template folder requirement: The university template suggests separate "Software implementation" and "Unit test implementation" folders. In this repository, **implementation and tests coexist in frontend/**, following standard Next.js practice:

- **Software implementation:** frontend/app/, frontend/components/, frontend/lib/
- **Unit tests:** co-located as *.test.ts files next to the modules they test (e.g., route.test.ts beside route.ts)

This approach improves maintainability because tests remain adjacent to the code they validate.

7.2.3 GitHub Repository

Repository URL: [https://github.com/HamzahDrawsheh/Intrenships_Platform]

Live deployment: <https://intrenshipsplatform-iota.vercel.app/>

The repository includes:

Table 8: GitHub Repo

Item	Location / Description
Source code	frontend/
Database schema	frontend/supabase/migrations/
Unit tests	5 Vitest files (18 test cases)
Documentation	README.md, CONTEXT_ENG/
Environment template	frontend/.env.example
Production env validator	frontend/scripts/check-production-env.mjs

7.2.4 README Summary

The root README.md provides:

1. **Project overview** — InternConnect Jordan connects AI/Data Science students with companies and supervisors.
2. **Tech stack** — Next.js, Supabase, OpenAI, SMTP/Resend.
3. **Installation steps:**

```
cd frontend
npm install
cp .env.example .env.local    # configure Supabase, OpenAI, email keys
npm run dev
```

4. **How to run tests:**

```
cd frontend
npm test
```

5. **How to deploy database migrations:**

```
cd frontend
npm run supabase:push
```

7.2.5 Key Implementation Modules

Frontend (role-based dashboards)

Table 9: Key Implementation Models

Role	Main routes
Student	/dashboard/student, /internships, /applications, /resume-builder
Company	/dashboard/company, /company/internships, /company/applications
Supervisor	/dashboard/supervisor, /supervisor/students, /supervisor/internship-reports
Admin	/admin/dashboard, /admin/users, /admin/onboarding-requests

Backend (27 API routes)

Representative routes:

- /api/applications/apply — Submit internship application
- /api/applications/commit — Confirm accepted placement
- /api/recommendations/internships — AI-ranked listings
- /api/chat/student-assistant — Student AI chatbot
- /api/company/applications/[applicationId]/cv — Secure signed CV URL
- /api/cron/maintenance — Background jobs (email queue, expiry automation)
- /api/health — Health check endpoint

Security implementation (three layers)

1. **Supabase Auth** — Session-based authentication
2. **Next.js middleware** — Role-based route allowlists
3. **PostgreSQL RLS** — Table-level authorization policies

8. Testing & Evaluation

This chapter evaluates the implemented system through software testing, security validation, and production-readiness assessment. Traditional ML metrics (accuracy, precision, recall) are not the primary evaluation criteria because the project is an operational web platform with AI-assisted features rather than a classification research model.

8.1 Evaluation Metrics

8.1.1 Software Quality Metrics

Table 10: Software Quality Metrics

Metric	Description	Target
ESLint status	Static code quality	Zero errors/warnings
Production build	TypeScript compile + Next.js build	Pass
Unit tests	Vitest authorization and parsing tests	All passing
Dependency audit	<code>npm audit</code> vulnerabilities	Zero high/critical
Migration deployment	Supabase schema push	Successful

8.1.2 Security Evaluation Metrics

Table 11: Security Evaluation Metrics

Metric	Description
Unauthorized API access	Non-admin blocked from admin/diagnostic routes (401/403)
CV access control	Only authorized company users receive signed CV URLs
Cron endpoint protection	Maintenance/queue routes require valid <code>CRON_SECRET</code>
Role escalation prevention	Non-admin cannot modify <code>profiles.role</code> or <code>is_suspended</code>
Rate limiting	AI routes throttled per user

8.1.3 Platform Functional Metrics (Post-Launch KPIs)

Table 12: Platform Functional Metrics

Metric	6-Month Target
Registered students	500+
Registered companies	40+
Active internship listings	80+
Total applications	1,000+
Student-to-placement conversion	$\geq 25\%$
Application-to-decision time	≤ 7 days

8.1.4 AI Feature Evaluation Criteria

Because AI outputs are assistive rather than autonomous decisions, evaluation focuses on:

Table 13: AI Feature Evaluation Criteria

Criterion	Method
Recommendation relevance	Semantic similarity score + skill-gap alignment
Response validity	JSON schema parsing tests (interview simulator)
Cost control	Rate limits and caching
User override	Final decisions remain with student/company/supervisor

8.2 Experimental and Validation Results

8.2.1 Production Readiness Scorecard

Table 14: Production Readiness Scorecard

Metric	Before Hardening	After Hardening
Project readiness score	72/100	84/100
Lint status	Passed with warnings	Clean
Unit tests	Minimal	5 files, 18 tests passing
Dependency vulnerabilities	6	0
Email diagnostics	Public risk	Admin-only
Cron queue processing	Unprotected if secret missing	Requires <code>CRON_SECRET</code>
Rate limiting	In-memory only	Supabase-backed shared limiter
Recommendation performance	Recalculation-heavy	Cached fallback

8.2.2 Comparison with Traditional Manual Workflow

Table 15: Comparison with Traditional Manual Workflow

Area	Traditional Workflow	InternConnect Jordan
Internship search	Informal channels	Centralized listings + AI recommendations
Applications	Email/manual forms	Structured workflow with status tracking
CV sharing	Uncontrolled email attachments	Secure signed URL access
Monthly reports	Paper/separate documents	In-platform digital workflow
Supervision	Limited visibility	Department-scoped supervisor dashboard
Notifications	Manual reminders	In-app + queued email notifications
Security	Minimal role control	Auth + middleware + RLS

9. Deployment

9.1 Deployment Architecture

InternConnect Jordan is deployed as a **serverless full-stack web application** using Vercel for the Next.js application and Supabase for backend services.

9.1.1 Deployment Components

Table 16: Deployment Components

Component	Platform	Role
Web application	Vercel	Hosts Next.js frontend + API routes
Database	Supabase PostgreSQL	Persistent data with RLS
Authentication	Supabase Auth	User sessions and email verification
File storage	Supabase Storage	CVs, logos, report PDFs
AI services	OpenAI API	Embeddings and LLM features
Email delivery	SMTP / Resend	Transactional notifications
Scheduled jobs	Vercel Cron	Calls <code>/api/cron/maintenance</code> with <code>CRON_SECRET</code>
Health monitoring	GET <code>/api/health</code>	Uptime checks

Live URL: <https://intrenshipsplatform-iota.vercel.app/>

9.1.2 Deployment Diagram

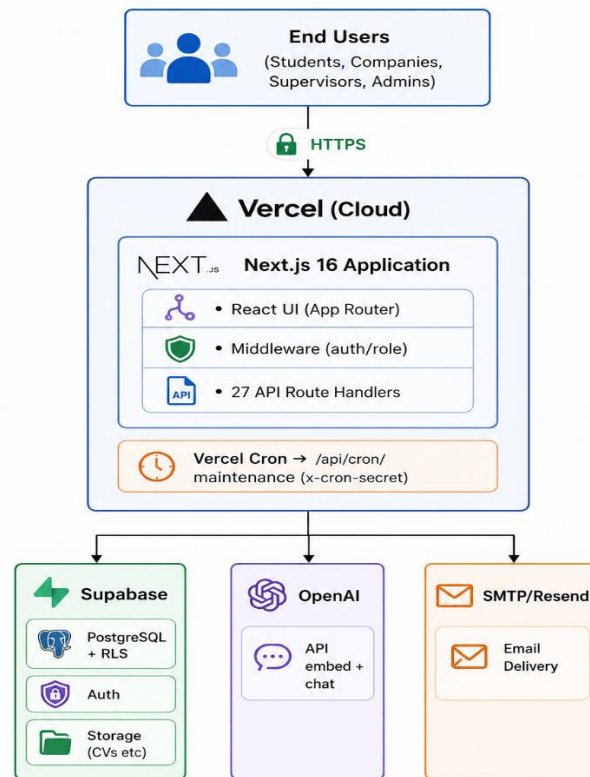


Figure 17: Deployment Diagram

9.2 Maintenance and Scalability

9.2.1 Ongoing Maintenance

Table 17: Ongoing Maintenance

Task	Method
Email queue processing	Scheduled <code>/api/cron/maintenance</code>
Expired commitment handling	Automated in maintenance cron
Training auto-completion	Database function + cron
Embedding refresh	Admin-triggered <code>/api/embeddings/generate</code>
Dependency updates	<code>npm audit</code> , periodic package upgrades
Schema changes	Versioned SQL migrations

9.2.2 Scalability Considerations

Table 18: Scalability Considerations

Area	Current Approach	Future Improvement
Recommendation search	In-app cosine similarity + cache	Dedicated vector DB or pgvector ANN index
AI cost	Rate limits + caching	Usage dashboards, budget alerts
Email delivery	Queued with retry tracking	Dead-letter queue monitoring
Testing	Unit tests on critical routes	E2E tests + staging environment
Monitoring	Health endpoint	Sentry, Datadog, or Axiom integration
Multi-university	Single platform instance	Per-institution configuration

9.2.3 Rollback Strategy

- Previous Vercel deployment builds remain available for instant rollback
- Database migrations avoid destructive column removal during launch
- Failed cron jobs can be disabled and run manually with `CRON_SECRET`
- Email failures inspectable via `transactional_email_queue.last_error`

10. Results, Discussion & Conclusion

10.1 Discussion

InternConnect Jordan successfully achieved its primary objective of building a centralized, secure, and AI-enhanced internship management platform for AI and Data Science students in Jordan. The project demonstrates that modern full-stack engineering—combined with cloud AI services—can address real academic workflow problems without requiring a traditional standalone machine-learning research pipeline.

10.1.1 Achievement of Objectives

Table 19: Achievement of Objectives

Objective	Status
Full-stack web platform	Achieved — Next.js + Supabase deployed on Vercel
Four-role support	Achieved — student, company, supervisor, admin
Secure auth and authorization	Achieved — Auth + middleware + RLS
Internship lifecycle management	Achieved — apply, commit, report, evaluate
AI-assisted features	Achieved — recommendations, chat, CV, cover letter, feedback analysis
Production readiness	Partially achieved — readiness score 84/100; staging/E2E pending

10.1.2 Comparison with Related Work

Unlike general job platforms (LinkedIn, Indeed), InternConnect Jordan integrates **academic supervision, monthly internship reports, and department-scoped supervisor access**, which are essential for university compliance but absent from commercial job boards.

Unlike learning management systems (Moodle), the platform provides **company-side applicant review, secure CV management, AI career tools, and structured application workflows** specialized for internships.

The hybrid recommendation approach—combining semantic embeddings with deterministic business rules—aligns with current best practices in responsible AI deployment, where automated suggestions support but do not replace human decisions.

10.1.3 AI Integration Results

AI services proved most valuable in:

- Reducing student effort (cover letters, CV suggestions, career chat)
- Improving discovery quality (semantic internship matching)
- Extracting structured value from unstructured text (report tasks → skills, feedback → sentiment)
- Supporting supervisors and companies with summarized insights

Rate limiting and caching were essential engineering complements to AI integration, preventing cost overruns in a multi-user web environment.

10.1.4 Limitations

- Vector search scalability limited without ANN indexing
- Test coverage focused on security-critical paths, not full E2E flows
- No separate staging environment in current deployment
- Full observability tooling (Sentry, structured logging) not yet integrated

10.2 Conclusion and Future Work

InternConnect Jordan demonstrates that a graduation project in Computer Information Systems can deliver a production-oriented SaaS platform combining software engineering, database security, and practical AI integration. The system replaces fragmented manual internship coordination with a structured digital workflow accessible to students, companies, supervisors, and administrators.

The project increased production readiness from approximately **72/100 to 84/100** through security hardening, automated testing, dependency remediation, and deployment checklists.

Future Work

1. **Expanded automated testing** — E2E tests with Playwright; RLS policy tests
2. **Staging environment** — Separate Supabase project for pre-production validation
3. **Observability** — Sentry error tracking, structured logging, uptime dashboards
4. **Scalable vector search** — pgvector ANN index or external vector database
5. **Mobile application** — Native iOS/Android clients for students and companies
6. **Multi-university support** — Configurable supervisor dashboards per institution
7. **Enhanced explainability** — Show students why specific internships were recommended
8. **Video interview integration** — Scheduling tools for remote screening
9. **Students Evaluation Score** — Evaluate the students based on their commitment and attendance

References

GitHub Repo: https://github.com/HamzahDrawsheh/Intrenships_Platform

Live URL: <https://intrenshipsplatform-iota.vercel.app>

Amazon Web Services. (n.d.). *What is a vector database?* <https://aws.amazon.com/what-is/vector-databases/>

Next.js. (2026). *Next.js documentation*. Vercel. <https://nextjs.org/docs>

OpenAI. (2026). *OpenAI API documentation*. <https://platform.openai.com/docs>

PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>

Supabase. (2026). *Supabase documentation*. <https://supabase.com/docs>

Vercel. (2026). *Vercel documentation*. <https://vercel.com/docs>

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP*. <https://doi.org/10.18653/v1/D19-1410>

Jordan University of Science and Technology. (2026). *Internship training regulations* [internal academic reference — insert if applicable].